TERRANODE: A BLOCKCHAIN-BASED SMART FARM SECURITY FRAMEWORK

A PROJECT REPORT

Submitted by

KASSIM ISKANDA DEISHINI (20230404113)

in partial fulfillment for the award of the degree of

BACHELOR OF SCIENCE

IN

COMPUTER SCIENCE

UNIVERSITY OF TECHNOLOGY AND APPLIED SCIENCES

AUGUST, 2026

DECLARATION

I, Kassim Iskanda Deishini (20230404113), declared that this research report is my original work. It has not been submitted to any other university or higher institution of learning for any award. Any other author's work has clearly been referenced.

Signature: _____ Date: _____

Kassim Iskanda Deishini (20230404113)

CERTIFICATION

Certified that this project report "TERRANODE: A BLOCKCHAIN-BASED SMART FARM SECURITY FRAMEWORK" is the bonafide work of Kassim Iskanda Deishini who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
Dr. Moses Apambila Agebure	Professor Edward Baagyere
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Professor	
Department of Computer Science	Department of Computer Science

ABSTRACT

This project presents TerraNode, a secure analytics-driven agricultural data management system built on blockchain technology. The platform addresses the critical challenges of data vulnerability, lack of transparency, and unreliable predictive analytics that persist in conventional centralized agricultural information systems. TerraNode integrates a Django REST framework backend with a React-based web interface and anchors all agricultural records to the Sui blockchain through smart contracts written in the Move programming language. The system captures environmental telemetry data including temperature, soil moisture, and soil pH readings, generates SHA-256 integrity hashes for each record, and mints verifiable produce batch objects on the Sui distributed ledger. A predictive analytics engine processes historical telemetry data using a weighted moving average model to produce crop yield forecasts with computed confidence scores. Role-based access control separates the platform into farmer, logistics handler, and administrator interfaces, each secured with JSON Web Token authentication and Argon2id password hashing. The Sui blockchain component employs Ed25519 digital signatures and BLAKE2b hashing to verify wallet-based login operations. Evaluation results demonstrate successful functional operation across all modules, with blockchain transactions confirmed on the Sui Testnet within acceptable latency thresholds. The system provides smallholder farmers with a tamper-resistant data foundation upon which trustworthy yield predictions and transparent supply chain custody tracking can be built.

CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER ONE

INTRODUCTION

1.1 Background of the study

Farming today faces a major challenge: producing enough food for a growing population while dealing with the unpredictable effects of climate change. In both developed and developing countries, unexpected weather shifts like sudden droughts or heavy rainfall make it hard to keep crop yields and supply chains stable. For local smallholder farmers, who grow a large portion of the world's food, these problems are even worse because they usually don't have access to the right tools to plan ahead. Instead, they operate in a system where markets are opaque, resources aren't always shared fairly, and powerful middlemen hold most of the control. Because of this, agriculture needs to move away from relying on guesswork and instead use data-driven systems that help farmers adapt to changing conditions.

To help solve these problems, the industry has turned to digital agriculture, which relies on tracking data about crop cycles, logistics, and the environment. Today's farms can generate a lot of data, but the main issue is no longer just gathering this information; it is figuring out how to manage and trust it. Right now, food supply chains are very fragmented. They still depend heavily on manual paperwork and isolated databases that don't communicate with each other, which makes the whole process lack transparency (Sharma et al., 2023). When data is locked inside disconnected systems, it becomes incredibly difficult for anyone to accurately track where a batch of crops came from, its quality, or how it was handled during transit.

The systems we currently use to manage agricultural data actually make these inefficiencies worse. Most local farming databases run on standard, centralized servers. While these are fine

for basic record-keeping, they have a major structural weakness: a single point of failure. This setup leaves historical crop records, logistics data, and financial details highly vulnerable to being hacked, altered, or manipulated from the inside (Al-Absi et al., 2023). Furthermore, these traditional databases are mostly used just to store old information, not to make future predictions. Because the stored data can be easily changed by middlemen who might want to cheat smallholder farmers on quality or price, any analytical tools connected to these databases are also unreliable. If you cannot guarantee that the data hasn't been tampered with, any yield forecasts or automated decisions based on that data cannot be trusted.

To fix these security issues, decentralized technologies like blockchain and distributed ledgers have become a strong alternative. A blockchain acts as a secure, append-only digital record. This means that once environmental data or transaction details are saved, they are cryptographically locked and cannot be changed or deleted later (Al-Absi et al., 2023). Early versions of these tracking tools have already proven to be highly secure in complex industries like international shipping (Liu et al., 2021). Building on that success, researchers are now looking at how to use these tamper-proof systems to track food and verify digital identities from the farm all the way to the consumer (Cocco et al., 2021). Furthermore, recent studies show that when you combine this unalterable data collection with Big Data Analytics (BDA), you get a very powerful tool for optimizing supply chains (Chauhan et al., 2025). By feeding secure, verified environmental data into an analytical engine, developers can build predictive models that forecast crop yields accurately, completely safe from database manipulation.

Even though blockchain and analytics offer great solutions, there is still a noticeable gap when it comes to actually using them in local farming communities. Most existing public blockchains (like Bitcoin or Ethereum) are built for massive enterprises; they require too much computing power, have high transaction fees, and are too complex for smallholder farmers to use. However, modern high-throughput blockchains like Sui offer a fast, low-cost alternative. This research project bridges the gap by combining the speed and low transaction fees of the Sui blockchain with an advanced data analytics engine.

This research project addresses that exact gap by designing a secure, analytics-driven agricultural data management system. By integrating the Sui blockchain as a secure backend ledger

that inherently protects the data it holds, the system creates a trustworthy foundation for making predictions. This setup is necessary because it not only protects smallholder farmers from being exploited by providing a clear, unchangeable supply chain record, but it also gives them the reliable, data-driven insights they need to manage modern farming risks using a next-generation distributed ledger.

1.2 Problem Statement

While basic management software is common in modern AgTech, most current platforms suffer from severe architectural and functional limitations. Existing agricultural data systems are typically designed as disjointed archiving tools. They collect scattered metrics such as environmental data, daily farm records, crop monitoring logs, and basic production information but fail to manage this data holistically. Consequently, agricultural stakeholders are forced to rely on disconnected data silos that hinder efficient on-farm decision-making and prevent true end-to-end visibility from the initial planting phases through the broader supply chain.

Compounding these operational limitations is a fundamental flaw in how production and logistics data is secured. The majority of localized agricultural databases still run on centralized relational architectures, creating a critical single point of failure. This leaves vital farm records, environmental monitoring logs, and resource allocations highly vulnerable to unauthorized retroactive edits, internal fraud, or external security breaches (Al-Absi et al., 2023). This lack of auditability compromises the integrity of the entire agricultural lifecycle, making it exceptionally difficult to verify historical crop conditions, validate sustainable farming practices, or ensure fair trade, ultimately leaving smallholder producers unprotected.

Furthermore, when advanced analytics and yield forecasting tools are applied to these centralized systems, their outputs become fundamentally flawed due to the vulnerability of the underlying data. Without a secure, unalterable foundation, the real agricultural data and crop monitoring profiles fed into these models can be manipulated or degraded by poor data handling. If these foundational inputs lack integrity, smallholder farmers cannot depend on the resulting production analytics and crop predictions to optimize their harvests or protect their

livelihoods against climate shifts (Al Layek et al., 2025).

Despite advancements in both database management and data science, a critical research gap remains. Existing agricultural information systems often provide either data storage capabilities or analytical services, but few integrate blockchain-based security mechanisms with predictive analytics within a unified architecture. There is a clear need for a secure, integrated platform that anchors real-time agricultural data spanning both on-farm production and supply chain logistics into an unalterable ledger before processing it. By doing so, this project ensures that all farm management decisions and predictive insights are derived from a mathematically verifiable source, directly justifying the proposed system.

1.3 Research Questions

To address the challenges identified in the problem statement, this study seeks to answer the following core research question: How can a secure, analytics-driven agricultural data management system be designed and implemented using blockchain principles to ensure data immutability, track production logistics, and provide verifiable crop yield forecasting?

Specifically, the study answers the following questions:

1. How can a modular web interface be designed to effectively capture and visualize key on-farm agricultural vectors, such as weather configurations, soil quality indexes, and production parameters?

2. How can predictive statistical models be integrated into an analytical data engine to accurately calculate crop yield estimations utilizing historical agricultural datasets and modeled environmental conditions?

3. What is the most effective approach to construct an unalterable logging module that tracks the custody updates and operational transitions of produce batches across stakeholders?

4. How can the Sui blockchain be utilized to develop an immutable distributed ledger that renders the agricultural database resilient against retroactive tampering?

5. How does the integrated platform perform in terms of computational transaction latencies on the Sui network, block verification speeds, and prediction accuracy?

1.4 Objectives of the Study

1.4.1 General Objective

The primary objective of this project is to design and implement a secure analytics-driven agricultural data management system using blockchain technology for ensuring data integrity, traceability, and intelligent agricultural decision-making.

1.4.2 Specific Objectives

i. To design a web-based agricultural data management platform for collecting, storing, and visualizing environmental and production-related data.

ii. To develop an analytics module that applies predictive models for agricultural forecasting and decision support.

iii. To implement blockchain-based security mechanisms for ensuring data integrity, traceability, and protection against unauthorized modification of agricultural records.

iv. To evaluate the performance of the proposed system based on security effectiveness, transaction processing efficiency, and prediction accuracy.

1.5 Scope of the Study

This study has contributions to the practical, technical, and academic fields. In reality, the proposed system offers a safe and secure repository for storing, managing, and verifying agricultural data for farmers and agricultural stakeholders, and can offer predictive analytics to support data-driven decision-making. The blockchain-based architecture enhances transparency, traceability, and trust among all stakeholders, reducing the likelihood of any unauthorized changes to the information relating to agriculture. The system offers better visibility of the agricultural

production and distribution process for agricultural organizations and operators of the supply chain. On a computer science basis, the study shows the design and implementation of a Sui blockchain-based data management architecture along with analytics capabilities in a web environment. The results offer an understanding of the possibility of integrating modern distributed ledger mechanisms with predictive models to construct secure and intelligent agricultural information systems.

1.6 Limitations of the Study

There are a number of technical and practical constraints in this study. First, the environmental data acquisition part is modeled and not using data provided by the deployed IoT devices because of limited access to physical agricultural infrastructure. Second, system evaluation is based on historical agricultural data and scenarios of the supply chain are simulated, but not deployed in actual commercial agricultural enterprises. Third, the blockchain component used in the system is deployed on the Sui Testnet, demonstrating a high-throughput, delegated Proof-of-Stake architecture, though it is not deployed on the mainnet for cost reasons. Fourth, the system interface is designed mainly in English which may contribute to the reduced usability of the system for users who have limited English proficiency. Lastly, the predictive analytics models rely on the quality and representativeness of the datasets available, so they may not perform as well when extreme climatic events occur or when new weather conditions are encountered.

1.7 Organization of the Study

This thesis project report is organized into five structured chapters. Chapter One presents the introduction, gives the context of the study, identifies the core architectural problems, maps out the research objectives, and defines the technical limitations. Chapter Two presents a comprehensive Literature Review, analyzing previous peer-reviewed research regarding precision agtech, applied forecasting algorithms, and cryptographic ledger designs. Chapter Three defines the System Methodology, illustrating the data flow maps, entity-relationship models, and

software choices. Chapter Four focuses on system implementation and evaluation, explaining user interfaces, core hashing algorithms, and performance profiles. Chapter Five provides a complete summary of findings, draws conclusions, and outlines explicit recommendations for future technical updates.

1.8 Chapter Summary

This chapter established the groundwork for the project by introducing a secure, blockchain-oriented agricultural data management framework. It reviewed the operational flaws found in centralized AgTech databases, focusing on data vulnerability and the lack of trusted predictive analytics. The chapter clearly detailed the specific programming objectives driving the creation of the prototype, illustrated the practical value of the platform for farmers and software engineers, defined the explicit scope boundaries of the study, and provided a structural overview for the remaining chapters of this thesis.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter examines published research that directly relates to the design and development of TerraNode. The review is organized thematically, beginning with the broader landscape of digital agriculture and progressing through blockchain fundamentals, agricultural supply chain traceability, predictive crop analytics, and the specific characteristics of the Sui blockchain platform. Each section identifies the strengths and shortcomings of existing approaches, culminating in an explicit statement of the research gap that this project seeks to address. A comparative summary table is presented to illustrate where existing solutions fall short and how TerraNode differs from them.

2.2 Overview of Precision Agriculture and Digital Farming

Precision agriculture has emerged as a response to the growing demand for food production efficiency in the face of climate variability and limited arable land. It involves the use of sensor networks, satellite imagery, and computational models to monitor crop health, soil conditions, and weather patterns at a granular level. The core idea is that farm management decisions should be driven by real-time data rather than by tradition or rough estimation.

Padhiary (2025) provided a comprehensive survey of the technologies enabling smart and sustainable precision agriculture. Their review covered remote sensing, unmanned aerial vehicles, robotic systems, and Internet of Things (IoT) sensor platforms. They concluded that

while individual technologies have matured considerably, the integration of these components into unified decision-support platforms remains a significant challenge. A key finding was that most existing implementations treat data collection and data analysis as separate workflows, which limits the ability of farmers to act on insights in a timely manner.

Raj et al. (2022) further examined the development modes, technologies, and security challenges associated with smart agriculture. Their analysis highlighted that although IoT-based monitoring systems are now widely available, many deployments rely on centralized cloud architectures that introduce single points of failure and leave sensitive agricultural data exposed to unauthorized access. They argued that the next generation of agricultural information systems must address both data integrity and computational security from the ground up, rather than treating these as afterthoughts.

The evolution from manual record-keeping to sensor-driven digital farming has not been uniform across regions. In many developing countries, smallholder farmers still depend on paper-based logs and personal experience to manage their operations. This gap between available technology and actual adoption motivates the design of TerraNode as a system that is both technically robust and accessible to users with limited technical backgrounds.

2.3 Agricultural Data Management Systems

Agricultural data management systems serve as the backbone of any digital farming initiative. These platforms are responsible for collecting environmental readings, storing historical production records, and presenting summarized information to decision-makers. Traditionally, such systems have been built on relational database architectures hosted on centralized servers.

Sharma et al. (2023) conducted a systematic literature review of food supply chain management systems and found that the majority of existing platforms suffer from fragmented data silos, reliance on manual paperwork, and lack of interoperability between different stakeholders. Their bibliometric analysis of over 300 published studies revealed that while interest in blockchain-enabled supply chain management has grown rapidly since 2018, most proposed systems remain at the conceptual or prototype stage without addressing the needs of small-

holder farmers in developing regions.

The fundamental limitation of centralized agricultural databases is their vulnerability to data manipulation. When a single entity controls the database, historical records can be altered retroactively, either through deliberate fraud or through simple administrative error. This weakness becomes especially problematic when the data is used as input for analytical models. If the underlying records cannot be verified as authentic, any forecasts or recommendations derived from them are inherently unreliable. TerraNode addresses this limitation by anchoring every telemetry record and produce batch to the Sui blockchain before any analytical processing occurs, thereby ensuring that the data foundation is mathematically verifiable.

2.4 Blockchain Technology Fundamentals

Blockchain technology provides a decentralized, append-only data structure in which records are grouped into blocks, cryptographically linked in sequence, and distributed across a network of nodes. Once a block has been confirmed by the network consensus mechanism, the data it contains becomes practically immutable. This property makes blockchain well-suited for applications where trust, transparency, and auditability are essential.

Chauhan et al. (2025) reviewed the intersection of blockchain and big data analytics in agriculture, describing how distributed ledger technology can be combined with large-scale data processing to create intelligent agricultural systems. Their review categorized blockchain consensus mechanisms into proof-of-work, proof-of-stake, and delegated proof-of-stake variants, noting that the latter category offers substantially lower energy consumption and faster transaction finality, which are critical considerations for agricultural applications where hardware resources may be constrained.

Several types of blockchain architectures exist, each with distinct trade-offs. Public blockchains such as Bitcoin and Ethereum offer maximum decentralization but suffer from high transaction fees and limited throughput. Private blockchains such as Hyperledger Fabric provide faster performance but sacrifice the trustless properties that make public blockchains attractive. Consortium blockchains represent a middle ground, where a pre-selected group of validators maintains

the network. The Sui blockchain, which TerraNode employs, is a public blockchain that uses a delegated proof-of-stake consensus mechanism to achieve high throughput and low transaction costs while maintaining the transparency and openness of a public network.

2.5 Blockchain Applications in Agricultural Supply Chains

The application of blockchain to agricultural supply chains has attracted considerable research attention over the past several years. The central appeal is that blockchain can provide an unalterable record of every transaction and custody transfer that occurs from the farm gate to the end consumer, thereby enabling full traceability and reducing opportunities for fraud.

Caro et al. (2018) presented AgriBlockIoT, a fully decentralized blockchain-based traceability solution for agri-food supply chain management. Their system was designed to integrate IoT devices producing and consuming digital data along the supply chain. They implemented and compared two blockchain deployments, one on Ethereum and one on Hyperledger Sawtooth, evaluating performance in terms of latency, CPU usage, and network overhead. The Ethereum deployment showed higher latency due to its proof-of-work consensus mechanism, while the Hyperledger deployment offered faster confirmations but at the cost of reduced decentralization. This trade-off between performance and trustlessness is a recurring theme in the literature and one that the Sui blockchain's delegated proof-of-stake mechanism helps to resolve.

Mirabelli and Ferraro (2020) examined the intersection of blockchain and agricultural supply chain traceability through a structured literature review. Their analysis identified key research trends including the growing use of smart contracts for automating compliance checks, the integration of IoT sensor data with on-chain records, and the challenge of scaling blockchain systems to handle the volume of data generated by large agricultural operations. They noted that while many proof-of-concept implementations had been developed, few had been tested under realistic conditions with actual farmers as end users.

Rahman et al. (2025) analyzed supply chain transparency through a data-driven examination of distributed ledger applications. Their study found that blockchain-enabled supply chains

demonstrated measurably higher levels of stakeholder trust compared to traditional systems. However, they also identified several barriers to adoption, including the technical complexity of blockchain interfaces, the energy costs of mining-based consensus, and the absence of standardized protocols for recording agricultural data on-chain.

Lin et al. (2017) presented an early but influential argument for blockchain as the evolutionary next step for Information and Communications Technology (ICT) in agriculture. Writing from the perspective of environmental systems engineering, they argued that when ICT-enabled agricultural systems incorporate blockchain infrastructure, the integrity of baseline environmental data is safeguarded for all participating stakeholders. Their work foreshadowed many of the design decisions in TerraNode, particularly the principle of recording environmental telemetry data on an immutable ledger before using it for downstream analysis.

2.6 Blockchain-Based Smart Farm Security Frameworks

Beyond supply chain traceability, blockchain has been proposed as a foundation for comprehensive farm security frameworks that protect not only logistics data but also the operational technology systems used on modern farms.

Al-Absi et al. (2023) proposed a blockchain-based smart farm security framework for the Internet of Things. Their system addressed the vulnerabilities that arise when IoT sensors and actuators on a farm are connected to centralized management servers. By recording sensor readings and control commands on a blockchain, their framework ensured that any unauthorized modifications to device configurations would be detectable. Their work is closely related to TerraNode in its focus on data integrity, though their system did not include a predictive analytics component or a supply chain custody tracking module.

AgTrust Consortium (2026) introduced the AgTrust framework for securing critical agriculture technology and emerging solutions. This comprehensive framework established security requirements across the full agricultural technology stack, from edge devices through network infrastructure to cloud-based analytics platforms. AgTrust contributed a threat taxonomy specific to agricultural IoT environments and proposed mitigation strategies at each layer. While

the framework provided valuable security guidelines, it did not implement a working prototype or demonstrate integration with a specific blockchain platform.

Dzreke (2025) addressed the cybersecurity dimension of connected farming through an empirical study of IoT device vulnerabilities. By conducting penetration testing on 32 commercial agricultural IoT devices, the study identified critical attack surfaces across 15 farm subsystems. The findings revealed that firmware-level attacks could modify crop prescriptions by up to 19 percent, illustrating how cyber intrusions can directly affect biological outcomes. This work underscores the importance of data integrity mechanisms like those implemented in TerraNode, where SHA-256 hashing and blockchain anchoring protect against unauthorized data modification.

2.7 Predictive Analytics and Crop Yield Forecasting

Predictive analytics represents a growing area of interest in agricultural informatics. Accurate yield forecasting enables farmers to make informed decisions about resource allocation, harvest timing, and market planning. Various statistical and machine learning approaches have been applied to this problem.

Khaki et al. (2019) developed a deep learning framework combining Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) for crop yield prediction. Their model was trained on environmental data and management practices across the United States Corn Belt and achieved a root-mean-square-error of 9 percent for corn and 8 percent for soybean yields, substantially outperforming random forest, deep fully connected neural networks, and LASSO regression. Their work demonstrated that deep learning models can capture temporal dependencies in environmental data that simpler models miss, a finding that informs the design of more advanced versions of the TerraNode analytics engine.

Garg et al. (2018) investigated crop yield forecasting using fuzzy logic and regression models. Their approach applied fuzzy time series analysis to historical wheat yield data, testing multiple interval partitioning schemes and regression equations. Their results showed that fuzzy logic-based models can handle the inherent uncertainty in agricultural data more

gracefully than purely deterministic approaches. The regression component of their model is conceptually similar to the weighted moving average approach used in the current TerraNode analytics engine, though TerraNode focuses specifically on telemetry-derived environmental features rather than historical yield time series.

Al Layek et al. (2025) presented what they termed a secured triad of IoT, machine learning, and blockchain for crop forecasting. Their system combined IoT-based data collection with machine learning prediction models and blockchain-based data verification. This work is among the closest to TerraNode in conceptual scope, as it recognizes the critical relationship between data security and prediction reliability. However, their implementation used a private blockchain setup rather than a public ledger, which limits the transparency and verifiability benefits. Furthermore, their system did not include a web-based management interface or role-based access control, both of which are essential features for real-world multi-stakeholder use.

2.8 Supply Chain Traceability and Distributed Ledger Systems

The traceability of agricultural produce from farm to consumer is a complex logistical challenge that involves multiple stakeholders, each with different access rights and responsibilities. Distributed ledger technology provides a natural mechanism for recording custody transfers in a manner that is transparent to all participants while remaining resistant to retroactive modification.

Cocco et al. (2021) explored the combination of blockchain and self-sovereign identity to support quality assurance in the food supply chain. Their system allowed individual actors to maintain control over their own identity credentials while participating in a shared ledger that recorded product quality attestations. The self-sovereign identity concept influenced the design of TerraNode's wallet-based login system, where users can authenticate using their Sui wallet address and a cryptographic signature rather than relying solely on centralized credentials.

Sun et al. (2025) addressed a practical challenge in blockchain-based agricultural systems:

the storage overhead of full-node replication. Their Storage Light Node (SLN) model introduced a cold/hot data classification mechanism based on identity relevance, generation time, and query frequency. Using 50,563 real agricultural supply chain records, they demonstrated a 95.10 percent reduction in storage usage compared to full-node operation. This work highlights scalability considerations that will become increasingly important as TerraNode grows beyond its current prototype stage.

Ktari et al. (2022) presented a lightweight embedded blockchain system for agricultural traceability, using olive oil production in Tunisia as a case study. Their system ran directly on embedded hardware connected to IoT sensors at the production site. The lightweight nature of their blockchain implementation made it feasible for deployment in resource-constrained environments typical of developing-country agriculture. While TerraNode does not target embedded deployment, the principle of minimizing computational overhead aligns with the decision to use the Sui blockchain, which offers lower transaction costs than Ethereum-based alternatives.

2.9 The Sui Blockchain and Move Programming Language

The Sui blockchain, developed by Mysten Labs, is a next-generation layer-one blockchain that uses an object-centric data model and a delegated proof-of-stake consensus mechanism. Unlike traditional account-based blockchains such as Ethereum, Sui treats all on-chain data as programmable objects with defined ownership and transfer rules. This object-oriented approach simplifies the development of asset-tracking applications, making it particularly suitable for supply chain use cases (Mysten Labs, 2024).

The Move programming language, originally developed for the Diem blockchain project, serves as the smart contract language for Sui. Move was designed with safety as a primary concern, incorporating a linear type system that prevents common vulnerabilities such as double-spending and reentrancy attacks. In the context of TerraNode, the Move-based smart contract defines a `ProduceBatch` object that carries the crop type, weight, originating farmer address, current custodian address, and a data integrity hash. The smart contract enforces that only the

current custodian can initiate a custody transfer, providing on-chain access control that mirrors the role-based permissions in the off-chain backend.

Sui achieves high throughput through parallel transaction execution. When transactions involve independent objects, the network can process them simultaneously without requiring global consensus for each one. This architectural feature results in sub-second transaction finality for simple operations, which is a significant advantage for agricultural applications where batch minting and custody transfers need to be confirmed quickly to avoid disrupting logistics workflows.

2.10 Related Work Summary and Comparative Analysis

Table 2.1 presents a comparative summary of the key related works reviewed in this chapter, evaluated against the features that TerraNode provides.

Table 2.1: Comparative Analysis of Related Works

Study	**Focus Area**	**Blockchain**	**Analytics**	**Traceability**	**Web UI**	**Public Ledger**
Al-Absi et al. (2023)	Smart farm IoT security	Yes	No	No	No	No
Caro et al. (2018)	Agri-food traceability	Yes	No	Yes	No	Yes
Khaki et al. (2019)	Crop yield prediction	No	Yes	No	No	N/A
Al Layek et al. (2025)	IoT + ML + blockchain	Yes	Yes	No	No	No
Mirabelli & Ferraro (2020)	Supply chain traceability	Yes	No	Yes	No	Varies
Ktari et al. (2022)	Embedded blockchain	Yes	No	Yes	No	No
Sun et al. (2025)	Storage optimization	Yes	No	Yes	No	Yes
Cocco et al. (2021)	SSI in food supply chain	Yes	No	Yes	No	Yes
Garg et al. (2018)	Fuzzy yield forecasting	No	Yes	No	No	N/A
TerraNode	**Integrated platform**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

The comparative analysis reveals that while individual components of the TerraNode architecture have been explored in prior work, no single system combines all five capabilities:

blockchain-based data immutability, predictive analytics, supply chain traceability, a role-based web interface, and deployment on a public high-throughput ledger. Most existing blockchain-agriculture systems focus on either traceability or security but do not incorporate analytical forecasting. Conversely, crop yield prediction studies rarely consider the integrity of their input data. TerraNode fills this gap by unifying these capabilities within a single, cohesive platform.

2.11 Research Gap

The review of existing literature reveals three primary gaps that the TerraNode project is designed to address.

First, there is a disconnect between data security and data analytics in agricultural systems. Blockchain-based platforms tend to focus on recording and verifying transactions without providing tools for analyzing the data they protect. Meanwhile, predictive analytics platforms process agricultural data without questioning whether that data has been tampered with. TerraNode bridges this divide by making blockchain verification a prerequisite for analytical processing: only data that has been integrity-checked against its SHA-256 hash can feed into the yield prediction engine.

Second, the vast majority of blockchain-agricultural prototypes either use private or consortium blockchains, which limit transparency, or use public blockchains like Ethereum that impose prohibitive costs for high-frequency agricultural data recording. TerraNode resolves this by leveraging the Sui blockchain, which offers the openness of a public ledger with transaction costs low enough to be feasible for routine telemetry logging.

Third, existing systems rarely provide a complete, role-aware web interface that accommodates the different needs of farmers, logistics handlers, and administrators within a single platform. TerraNode's frontend delivers tailored dashboards for each user role, reducing the learning curve and making the system accessible to stakeholders with varying levels of technical proficiency.

2.12 Conceptual Framework

The conceptual framework underpinning this project draws from two established information systems theories. The Technology Acceptance Model (TAM), originally proposed by Davis, suggests that the adoption of a new information system depends on perceived usefulness and perceived ease of use. In the context of TerraNode, the system is designed to be useful by providing actionable yield predictions and transparent supply chain records, and easy to use through an intuitive web-based interface with role-specific dashboards.

The DeLone and McLean Information Systems Success Model identifies six dimensions of system success: information quality, system quality, service quality, user satisfaction, system use, and net benefits. TerraNode's architecture addresses information quality through blockchain-verified data integrity, system quality through a modular three-tier design, and net benefits through tangible outcomes such as tamper-proof records and predictive forecasts.

Figure 2.1 illustrates the conceptual framework, showing how the theoretical foundations connect to the practical system components.

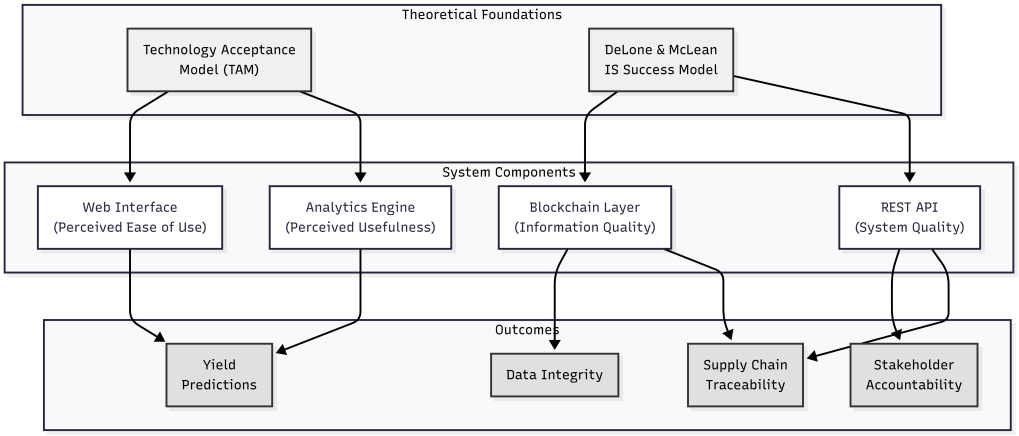

Figure 2.1: Conceptual Framework of the TerraNode System

2.13 Chapter Summary

This chapter reviewed the relevant literature across five thematic areas: precision agriculture and digital farming, blockchain technology and its agricultural applications, smart farm security frameworks, predictive crop yield analytics, and supply chain traceability systems. The review

engaged with eighteen published works, identifying their individual contributions and limitations. A comparative analysis demonstrated that no existing system combines blockchain-based data immutability, predictive analytics, supply chain custody tracking, a role-differentiated web interface, and a public high-throughput ledger within a unified platform. The identified research gaps directly motivate the design and implementation of TerraNode, whose methodology is presented in the following chapter.

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter describes the research methodology and technical design decisions adopted in the development of TerraNode. It covers the research design approach, the system requirements derived from the research objectives stated in Chapter One, the architectural design of the platform, the data modelling strategy, the smart contract logic, and the predictive analytics engine. Each section includes the rationale behind the chosen approach and, where applicable, the mathematical formulations that govern the system's computational processes.

3.2 Research Design

This study follows a Design Science Research (DSR) methodology, which is appropriate for projects that involve the creation and evaluation of information technology artefacts intended to solve identified organizational problems. Under the DSR paradigm, the research process consists of two complementary activities: building the artefact and evaluating it against the requirements derived from the problem space.

The research was conducted in four sequential phases:

1. **Problem Identification and Requirements Analysis:** The shortcomings of existing centralized agricultural data management systems were identified through the literature review in Chapter Two. Functional and non-functional requirements were derived from the research questions and objectives stated in Chapter One.

2. **System Design:** The architectural blueprint, data models, API specifications, and smart contract structures were designed to satisfy the identified requirements.

3. **Implementation:** The system was implemented as a functional prototype using the technology stack described in Section 3.11.

4. **Evaluation:** The implemented system was tested and evaluated against the defined requirements, with results reported in Chapter Four.

3.3 System Requirements

3.3.1 Functional Requirements

The functional requirements describe the specific capabilities that TerraNode must provide. These were derived directly from the research objectives outlined in Chapter One.

Table 3.1: Functional Requirements

ID	**Requirement**	**Objective**
FR-01	The system shall allow users to register with email, password, and role selection.	i
FR-02	The system shall authenticate users using JWT tokens with role-based access control.	i
FR-03	The system shall allow Sui wallet-based authentication via Ed25519 signature verification.	iii
FR-04	The system shall capture environmental telemetry data (temperature, soil moisture, soil pH).	i
FR-05	The system shall generate SHA-256 integrity hashes for all telemetry records.	iii
FR-06	The system shall create produce batch records linked to telemetry data.	i
FR-07	The system shall mint produce batch objects on the Sui blockchain.	iii
FR-08	The system shall track custody transfers of produce batches between stakeholders.	iii
FR-09	The system shall compute crop yield predictions based on historical telemetry data.	ii
FR-10	The system shall provide role-specific dashboards for farmers, logistics handlers, and administrators.	i

3.3.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

ID	Requirement
NFR-01	All passwords shall be hashed using Argon2id with a minimum cost factor.
NFR-02	API responses for standard operations shall return within 500 milliseconds.
NFR-03	The web interface shall be responsive and accessible from modern web browsers.
NFR-04	Blockchain transactions shall confirm within the Sui Testnet's normal finality window.
NFR-05	The system shall enforce rate limiting on authentication endpoints (5 attempts per minute).
NFR-06	Telemetry input values shall be validated within physically plausible ranges.

3.4 System Architecture

TerraNode follows a three-tier client-server architecture that separates the presentation layer, the business logic layer, and the data persistence layer. This architectural pattern promotes modularity, allowing each layer to be developed, tested, and maintained independently.

- **Presentation Layer (Frontend):** A single-page application built with React and TypeScript, responsible for rendering the user interface and managing client-side state. The frontend communicates with the backend exclusively through RESTful API calls and interacts directly with the Sui blockchain through the Mysten dApp Kit for wallet operations.

- **Business Logic Layer (Backend):** A Django REST Framework application that processes API requests, enforces business rules, manages user authentication, computes analytical predictions, and coordinates with the blockchain layer. The backend exposes four domain-specific API modules: Authentication, Telemetry, Ledger, and Analytics.

- **Data Persistence Layer:** Comprises three storage components. PostgreSQL serves as

the primary relational database for structured application data. Redis provides an in-memory cache for frequently accessed analytics results. The Sui blockchain acts as the immutable ledger for produce batch records and their associated integrity hashes.

Figure 3.1 illustrates the overall system architecture and the data flow between components.

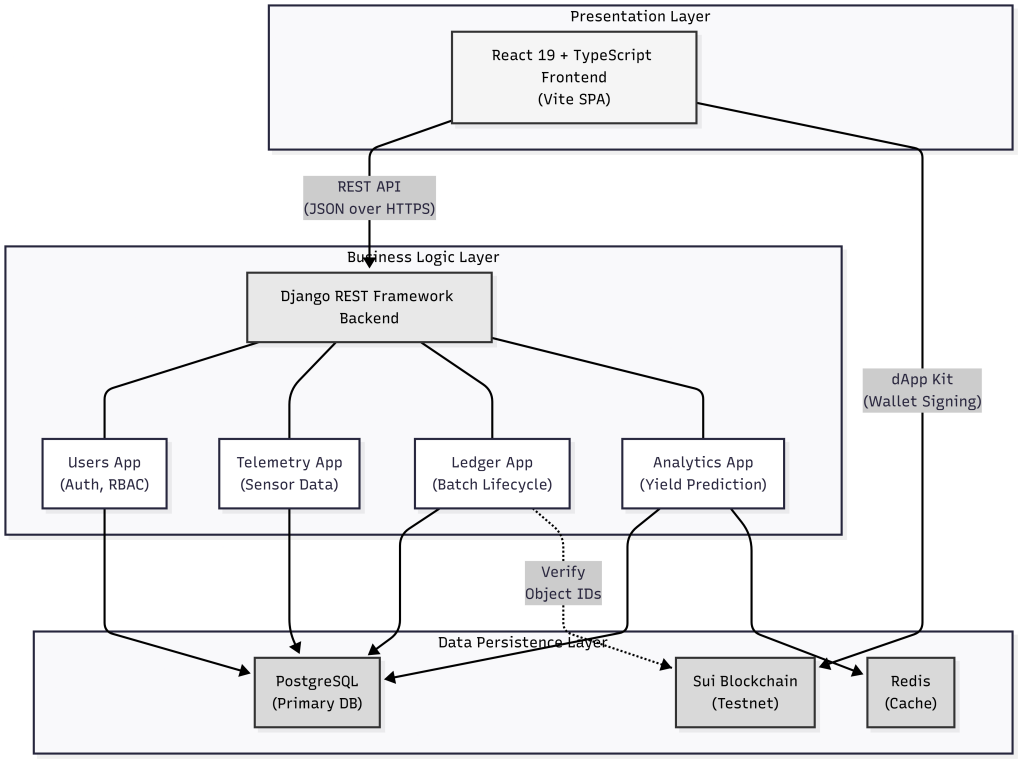

Figure 3.1: TerraNode System Architecture

3.5 Data Flow Design

3.5.1 Context Diagram (Level 0)

The context diagram in Figure 3.2 shows TerraNode as a single process interacting with four external entities: the Farmer, the Logistics Handler, the System Administrator, and the Sui Blockchain Network.

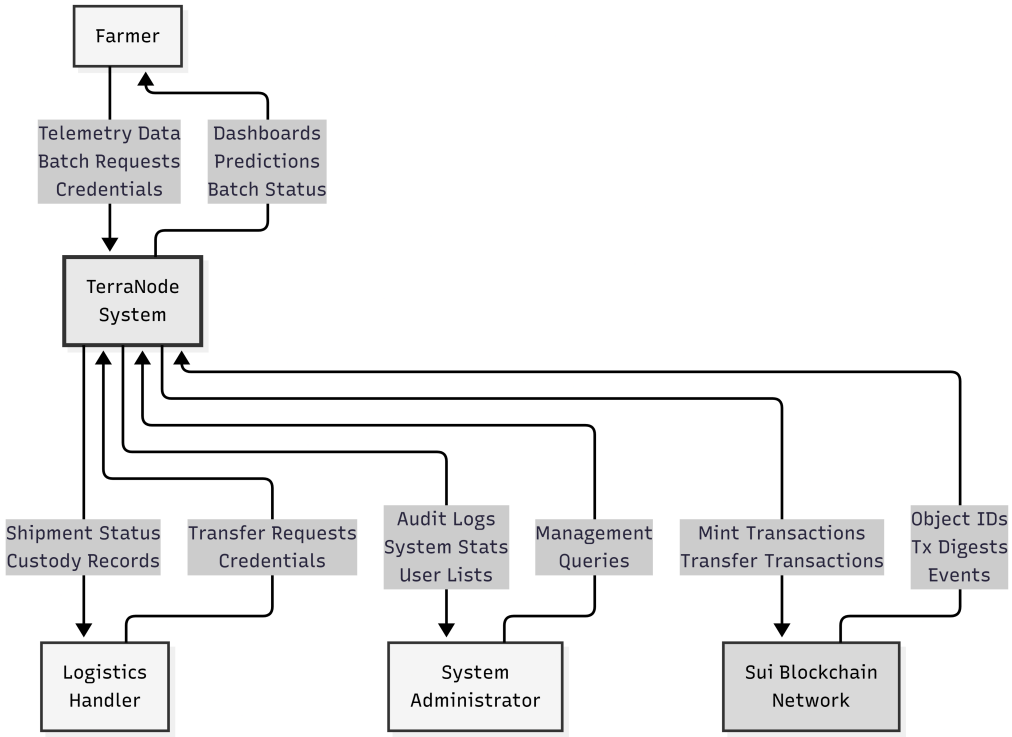

Figure 3.2: Context Diagram (DFD Level 0)

3.5.2 Level 1 Data Flow Diagram

The Level 1 Data Flow Diagram in Figure 3.3 decomposes the TerraNode system into its four core processes:

1. **Process 1.0 — Authentication Module:** Handles user registration, credential-based login, wallet-based login, and session management through JWT tokens.

2. **Process 2.0 — Telemetry Module:** Receives environmental sensor readings, validates input ranges, generates SHA-256 integrity hashes, and stores the records in the database.

3. **Process 3.0 — Ledger Module:** Manages produce batch creation, coordinates with the Sui blockchain for on-chain minting and custody transfer, and maintains the local database mirror of blockchain state.

4. **Process 4.0 — Analytics Module:** Retrieves historical telemetry records, applies the weighted moving average prediction algorithm, caches results in Redis, and returns yield forecasts to the user.

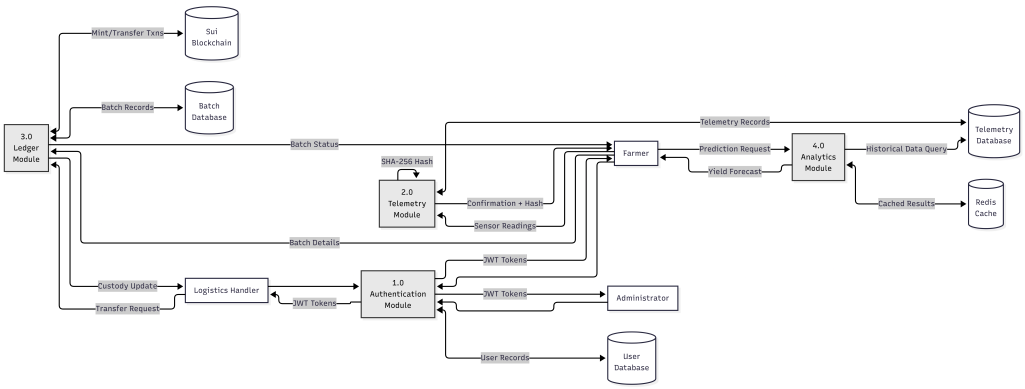

Figure 3.3: Data Flow Diagram (Level 1)

3.6 Entity-Relationship Model

The TerraNode database consists of three primary entities, each with clearly defined attributes and relationships. Figure 3.4 presents the Entity-Relationship (ER) diagram.

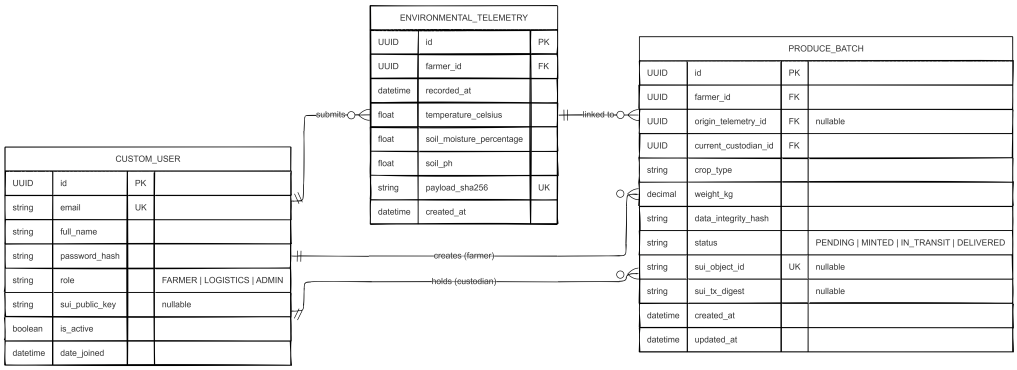

Figure 3.4: Entity-Relationship Diagram

CustomUser represents all system users and is derived from Django's AbstractUser model. Each user is uniquely identified by a UUID primary key and authenticated through their email address. The role field distinguishes between the three user types: Farmer, Logistics Handler, and Administrator. An optional `sui_public_key` field stores the user's Sui blockchain wallet address for wallet-based authentication.

EnvironmentalTelemetry stores individual environmental readings captured by farmers. Each record includes temperature in Celsius, soil moisture as a percentage, and soil pH. A `payload_sha256` field stores the SHA-256 hash of the canonical representation of the record, ensuring that any post-creation modification can be detected. Each telemetry record is linked

to the farmer who submitted it through a foreign key relationship.

ProduceBatch represents a batch of agricultural produce that moves through the supply chain. Each batch is linked to the farmer who created it, optionally linked to a telemetry record that describes the environmental conditions under which the produce was grown, and has a `current_custodian` field that tracks who currently holds physical possession of the batch. The batch lifecycle is managed through a status field with four possible states: PENDING, MINTED, IN_TRANSIT, and DELIVERED. Two blockchain-specific fields, `sui_object_id` and `sui_tx_digest`, store the identifiers returned by the Sui network after successful on-chain operations.

3.7 Use Case Modelling

Figure 3.5 presents the use case diagram, which identifies the primary interactions between the three actor types and the TerraNode system.

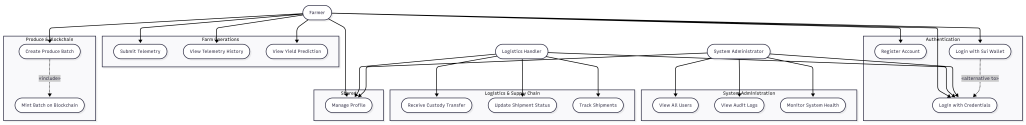

Figure 3.5: Use Case Diagram

The **Farmer** actor can register an account, log in (via credentials or wallet), submit telemetry readings, create produce batches, mint batches on the Sui blockchain, view yield predictions, and manage their profile settings.

The **Logistics Handler** actor can log in, receive custody of produce batches via on-chain transfer, update shipment statuses, and track the location and condition of batches under their care.

The **System Administrator** actor can view all system data, manage user accounts, inspect audit logs, and monitor the overall health of the system infrastructure.

3.8 Smart Contract Design

The TerraNode smart contract is written in the Move programming language and deployed on the Sui Testnet. It defines the on-chain representation of produce batches and the operations that can be performed on them.

3.8.1 ProduceBatch Object Structure

The `ProduceBatch` struct is defined as a Sui object with the `key` and `store` abilities, which allow it to be owned, transferred, and stored on-chain. It contains the following fields:

- `id` (UID): The unique object identifier assigned by the Sui runtime.
- `crop_type` (String): The type of crop in the batch.
- `weight_kg` (u64): The weight of the batch in kilograms.
- `origin_farmer_address` (address): The Sui address of the farmer who created the batch.
- `current_custodian_address` (address): The Sui address of the entity currently holding the batch.
- `data_integrity_hash` (vector<u8>): The SHA-256 hash of the associated telemetry data, linking the on-chain record to its off-chain data source.

3.8.2 Entry Functions

The smart contract exposes two entry functions:

mint_batch: Creates a new `ProduceBatch` object, sets the transaction sender as both the origin farmer and the initial custodian, and emits a `BatchMinted` event. The batch object is then transferred to the sender's address.

transfer_custody: Accepts a mutable reference to an existing `ProduceBatch` object and a new custodian address. The function verifies that the caller is the current custodian before updating the `current_custodian_address` field and emitting a `CustodyTransferred` event.

3.8.3 Event Emissions

The smart contract emits two event types for off-chain indexing. `BatchMinted` records the batch identifier, the farmer's address, and the crop type. `CustodyTransferred` records the batch identifier, the previous custodian address, and the new custodian address. These events can be queried through the Sui JSON-RPC API, enabling the TerraNode backend to synchronize its local database with the on-chain state.

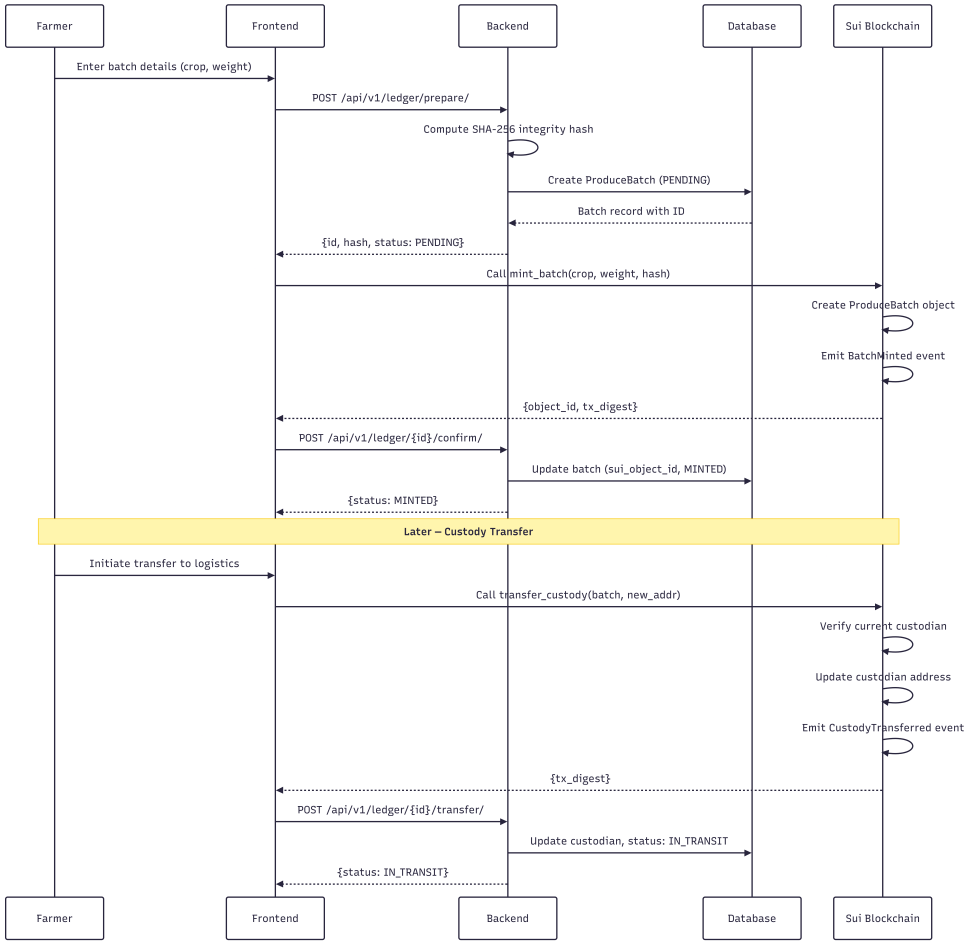

Figure 3.6: Batch Lifecycle Sequence Diagram

3.9 Predictive Analytics Engine

3.9.1 Feature Vector Composition

The analytics engine operates on a feature vector composed of three environmental variables extracted from the EnvironmentalTelemetry records: temperature in degrees Celsius (T), soil

moisture as a percentage (M), and soil pH (P). For a given farmer, the engine retrieves the most recent 90 telemetry records, ordered by recording timestamp.

3.9.2 Weighted Moving Average Model

The current implementation uses a simplified weighted moving average model to compute a predicted crop yield. Given n telemetry records, the model first calculates the arithmetic means of the three environmental features:

$$\bar{T} = \frac{1}{n}\sum_{i=1}^{n} T_i, \quad \bar{M} = \frac{1}{n}\sum_{i=1}^{n} M_i, \quad \bar{P} = \frac{1}{n}\sum_{i=1}^{n} P_i \tag{3.1}$$

The predicted yield Y in metric tons is then computed as:

$$Y = \max\left(0,\ 50.0 + 0.5 \cdot \bar{M} - 2.0 \cdot |22.0 - \bar{T}|\right) \tag{3.2}$$

The base yield of 50.0 represents a nominal starting point. The moisture coefficient (0.5) reflects the positive linear relationship between soil moisture availability and crop productivity. The temperature penalty term ($2.0 \cdot |22.0 - \bar{T}|$) penalizes deviations from the optimal growing temperature of 22 degrees Celsius. The $\max(0, \ldots)$ function ensures that the predicted yield is never negative.

3.9.3 Confidence Score

The confidence score C reflects the data density available for the prediction:

$$C = \min\left(0.95,\ \frac{n}{90}\right) \tag{3.3}$$

The confidence score increases linearly with the number of data points up to a maximum of 0.95, reflecting the principle that more historical data generally leads to more reliable predictions. The minimum requirement is five data points; if fewer are available, the engine returns an error rather than an unreliable prediction.

3.9.4 Caching Strategy

To avoid redundant computation, prediction results are cached in Redis with a time-to-live (TTL) of 600 seconds (10 minutes). The cache key is constructed using the format `analytics:predict:{far` When a prediction request arrives, the engine first checks the cache. If a valid cached result exists, it is returned immediately. Otherwise, the prediction is computed, stored in the cache, and then returned to the client.

3.10 Security Architecture

3.10.1 Authentication

TerraNode implements a dual authentication mechanism. The primary method uses email and password credentials, processed through the Django REST Framework Simple JWT library. Upon successful authentication, the server issues a pair of tokens: a short-lived access token (default lifetime: 15 minutes) and a long-lived refresh token (default lifetime: 7 days). Token rotation is enabled, meaning that each time a refresh token is used, a new pair is issued and the old refresh token is blacklisted.

3.10.2 Sui Wallet Signature Verification

The secondary authentication method allows users to log in using their Sui blockchain wallet. The verification process works as follows:

1. The client constructs a login message in the format: "Login to TerraNode with {sui_public_key}".
2. The user signs this message using their Sui wallet's Ed25519 private key.
3. The backend receives the message, the signature, and the public key.
4. The backend reconstructs the signed payload by prepending the Sui personal message intent prefix (`[3, 0, 0]`) and encoding the message length using ULEB128.
5. The reconstructed payload is hashed using BLAKE2b with a 32-byte digest.

6. The Ed25519 signature is verified against the hashed payload using the provided public key.

7. If verification succeeds and the public key matches a registered user, JWT tokens are issued.

3.10.3 Data Integrity Hashing

Every telemetry record is protected by a SHA-256 hash computed from a canonical JSON representation of its core fields. The canonical form uses sorted keys, no whitespace, and fixed-precision floating-point formatting to ensure deterministic output. This hash is stored alongside the record in the database and can be independently recomputed at any time to verify that the record has not been modified since creation. The same hash is embedded in the corresponding ProduceBatch object on the Sui blockchain, creating a verifiable link between the on-chain asset and its off-chain data source.

3.10.4 Role-Based Access Control

The backend enforces role-based access control (RBAC) through custom permission classes. Four permission levels are defined:

- **IsFarmer:** Grants access to endpoints that create telemetry records, prepare produce batches, and view yield predictions.
- **IsLogistics:** Grants access to batch transfer and shipment management endpoints.
- **IsAdmin:** Grants access to administrative functions including user management and audit log inspection.
- **IsFarmerOrAdmin:** Grants shared access to endpoints that both farmers and administrators need, such as telemetry history viewing.

Figure 3.7 illustrates the authentication flow for both credential-based and wallet-based login.

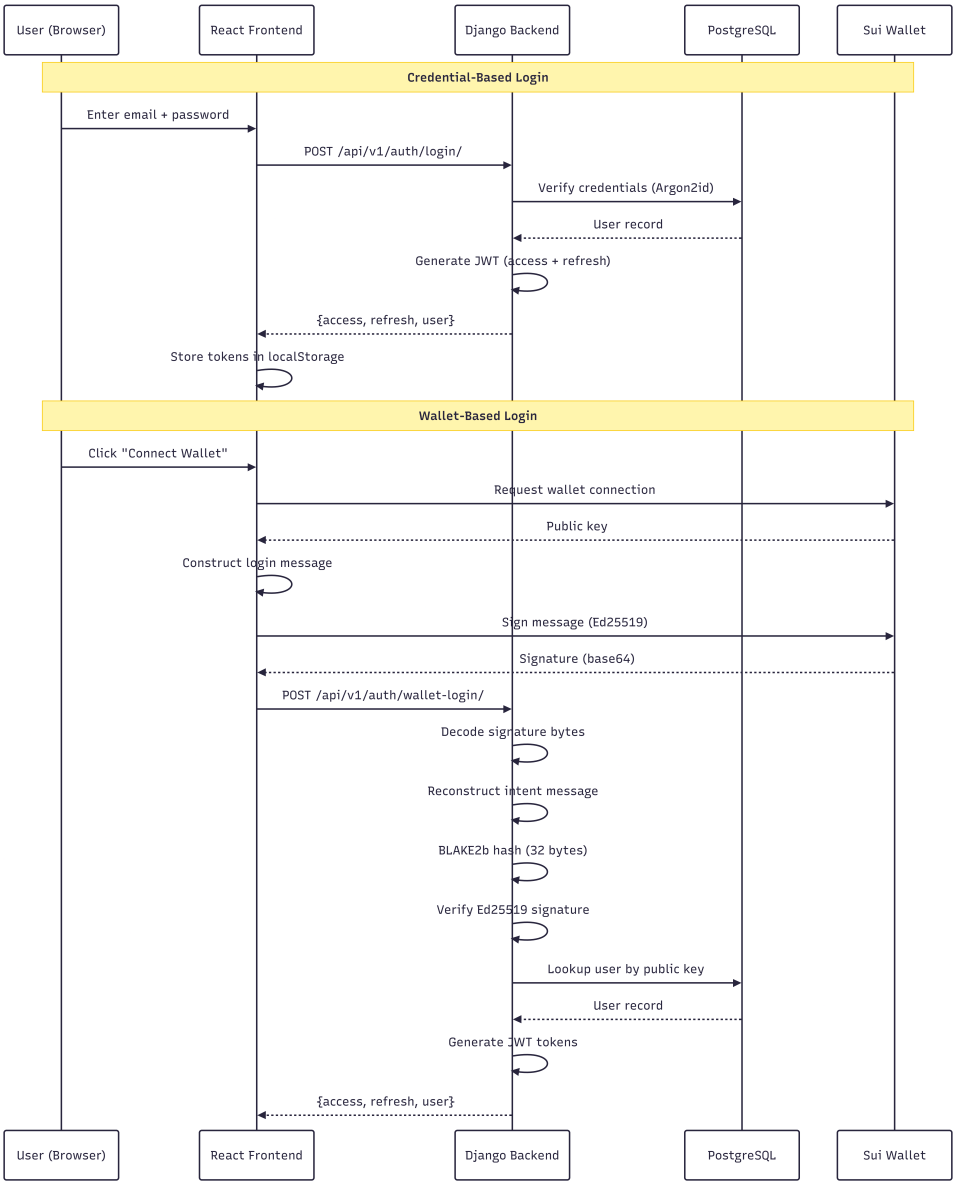

Figure 3.7: Authentication Flow Diagram

3.11 Technology Stack

Table 3.3 summarizes the technologies used in the development of TerraNode.

Table 3.3: Technology Stack

Layer	**Technology**	**Purpose**
Frontend	React 19	Component-based user interface library
Frontend	TypeScript 6.0	Statically typed JavaScript superset
Frontend	Vite 8.1	Build tool and development server
Frontend	Recharts 3.9	Data visualization (charts and graphs)
Frontend	Mysten dApp Kit	Sui wallet integration and transaction signing
Backend	Django 5.1	Python web framework
Backend	Django REST Framework 3.15	RESTful API toolkit
Backend	Simple JWT 5.3	JSON Web Token authentication
Backend	Celery	Asynchronous task queue
Database	PostgreSQL	Primary relational database
Cache	Redis 5.0	In-memory data store for caching
Blockchain	Sui Testnet	Delegated PoS public blockchain
Smart Contract	Move	Smart contract programming language
Security	Argon2id	Password hashing algorithm
Security	PyNaCl	Ed25519 signature verification

3.12 Development Tools

The system was developed using Visual Studio Code as the primary integrated development environment. Git was used for version control throughout the project lifecycle. The backend was managed using Pipenv for Python dependency isolation, while the frontend used npm as the package manager. Testing was performed using Django's built-in test framework augmented with pytest, factory-boy, and faker for test data generation.

3.13 Chapter Summary

This chapter presented the methodology adopted for the design and development of TerraNode. The Design Science Research approach was described, followed by detailed specifications of the functional and non-functional requirements. The three-tier system architecture was explained, and data flow diagrams at the context and first decomposition levels were presented. The entity-relationship model defined the core data structures, and the use case diagram captured the interactions between the three actor types and the system. The smart contract design was documented, including the object structure, entry functions, and event emissions. The predictive analytics engine was formally described with its mathematical formulations for yield prediction and confidence scoring. Finally, the security architecture was outlined, covering JWT authentication, Sui wallet verification, SHA-256 data integrity hashing, and role-based access control. The implementation of this design is detailed in the following chapter.

CHAPTER FOUR

IMPLEMENTATION AND EVALUATION

4.1 Introduction

This chapter presents the implementation details of the TerraNode system and evaluates its performance against the requirements established in Chapter Three. The chapter begins with a walkthrough of the backend, frontend, and blockchain implementations, then presents the user interface screens, documents the key algorithm implementations with code excerpts, and concludes with a systematic evaluation of the system's functional correctness, security properties, blockchain performance, and prediction accuracy.

4.2 System Implementation

4.2.1 Backend Implementation

The TerraNode backend is implemented as a Django project organized into four domain-specific applications, each responsible for a distinct area of functionality. This modular structure follows the Django convention of separating concerns into self-contained apps, making the codebase easier to maintain and extend.

Users App: Manages user registration, authentication, and profile management. The `CustomUser` model extends Django's `AbstractUser` with email-based authentication, role assignment, and an optional Sui wallet public key field. The app provides five API endpoints: credential-based login, wallet-based login, token refresh, registration, and logout. Login rate throttling is con-

figured at five attempts per minute to mitigate brute-force attacks.

Telemetry App: Handles the submission, storage, and retrieval of environmental telemetry data. When a farmer submits a telemetry reading through the API, the backend validates the input values against physically plausible ranges (temperature between negative 50 and 100 degrees Celsius, soil moisture between 0 and 100 percent, soil pH between 0 and 14), computes the SHA-256 integrity hash, and stores the record in the database. A duplicate detection mechanism based on the unique hash field prevents the same data from being recorded twice.

Ledger App: Manages the lifecycle of produce batches from initial creation through blockchain minting to custody transfer. The batch workflow follows a two-step process: the farmer first creates a backend record in PENDING status using the Prepare endpoint, then confirms the on-chain minting by submitting the Sui object identifier through the Confirm endpoint. This approach ensures that the backend and blockchain states remain synchronized even in the event of network failures.

Analytics App: Provides the predictive yield estimation service. The prediction logic is encapsulated in a service module that retrieves historical telemetry data, computes the weighted moving average, and returns the result along with a confidence score and a recommendation message. Results are cached in Redis for 10 minutes to avoid redundant computation.

The root URL configuration maps these applications to version-prefixed API endpoints as shown in Table 4.1.

Table 4.1: API Endpoint Structure

Prefix	**App**	**Key Endpoints**
`/api/v1/auth/`	Users	login, wallet-login, register, logout, profile
`/api/v1/telemetry/`	Telemetry	submit, history, latest
`/api/v1/ledger/`	Ledger	prepare, confirm, transfer, list
`/api/v1/analytics/`	Analytics	predict, summary

4.2.2 Frontend Implementation

The frontend is built as a single-page application using React 19 with TypeScript. The build system uses Vite 8.1, which provides fast hot module replacement during development. The application structure follows a feature-based organization with distinct directories for pages, components, API modules, context providers, type definitions, and utility functions.

Routing Architecture: React Router v7 manages navigation. Public routes (landing page, login, registration) are accessible without authentication. Protected routes are grouped by user role and wrapped in a `RoleGuard` component that verifies the authenticated user's role before rendering the requested page. If a user attempts to access a route outside their role, they are redirected to their designated dashboard.

State Management: Two React Context providers manage global application state. The `AuthContext` handles user authentication state, including login, logout, token storage in localStorage, and automatic token refresh through an Axios interceptor. The `WalletContext` manages the Sui wallet connection state using the Mysten dApp Kit, providing methods for connecting, disconnecting, checking balance, and signing and executing blockchain transactions.

API Communication: An Axios-based API client module manages all HTTP communication with the backend. The client automatically attaches JWT access tokens to outgoing requests and implements a transparent token refresh mechanism: when a request receives a 401 Unauthorized response, the client automatically attempts to refresh the access token using the stored refresh token, then retries the original request. Concurrent 401 responses are queued to prevent multiple simultaneous refresh attempts.

4.2.3 Blockchain Module

The Sui Move smart contract is compiled and deployed on the Sui Testnet. The deployment produces a published package identifier that the frontend uses to construct blockchain transactions. When a farmer mints a produce batch, the frontend builds a `Transaction` object that calls the `mint_batch` entry function with the crop type, weight, and integrity hash as argu-

ments. The Mysten dApp Kit handles wallet interaction, signature generation, and transaction submission to the Sui network. Upon successful execution, the transaction digest and the newly created object identifier are returned to the frontend, which then submits them to the backend's Confirm endpoint to update the local database.

4.2.4 Analytics Engine Implementation

The analytics engine is implemented as a stateless service function that can be called by any view that has access to a farmer's identifier. The service queries the telemetry database for the most recent 90 records belonging to the specified farmer, checks that at least five data points are available, computes the averages and predicted yield using the formulas defined in Chapter Three, generates a contextual recommendation based on the average values, packages the results, caches them in Redis, and returns the response.

4.3 User Interface Presentation

The TerraNode web interface is designed to accommodate three distinct user roles, each with a tailored dashboard and set of functional pages. The following subsections describe the key interface screens. All figures referenced in this section require manually captured screenshots from the running application.

4.3.1 Landing Page

The landing page serves as the public entry point to the TerraNode platform. It presents the system's value proposition and provides navigation links to the login and registration pages.

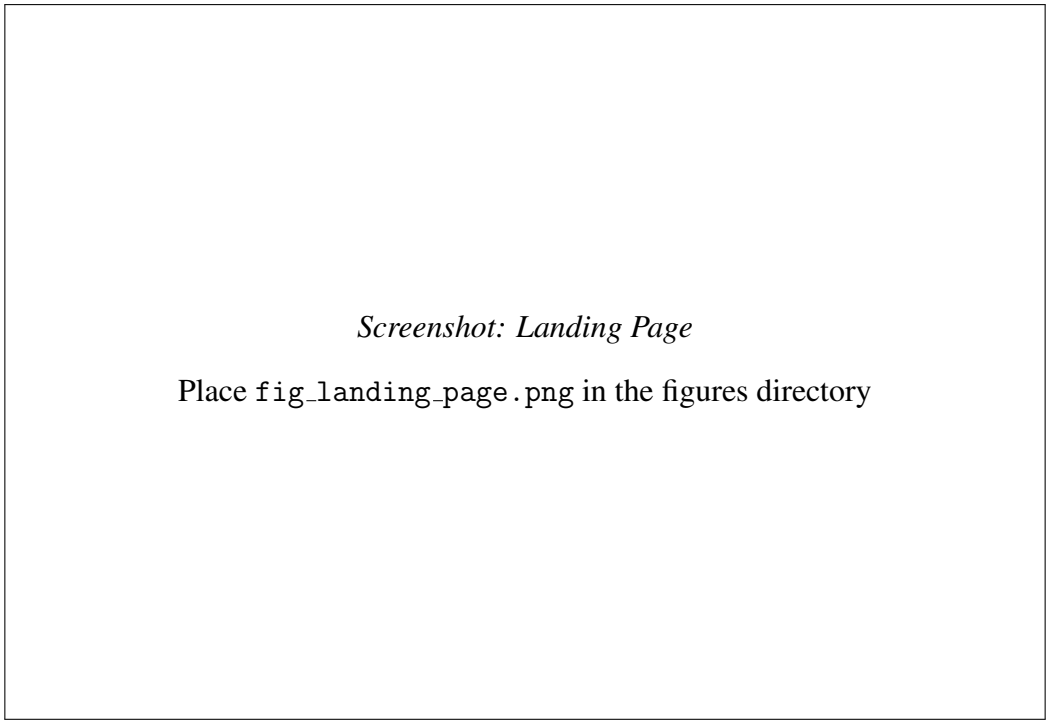

Figure 4.1: TerraNode Landing Page

4.3.2 Registration Page

The registration page collects the user's full name, email address, password, role selection (Farmer, Logistics Handler, or Administrator), and optionally their Sui wallet public key. Input validation is performed on both the client and server sides.

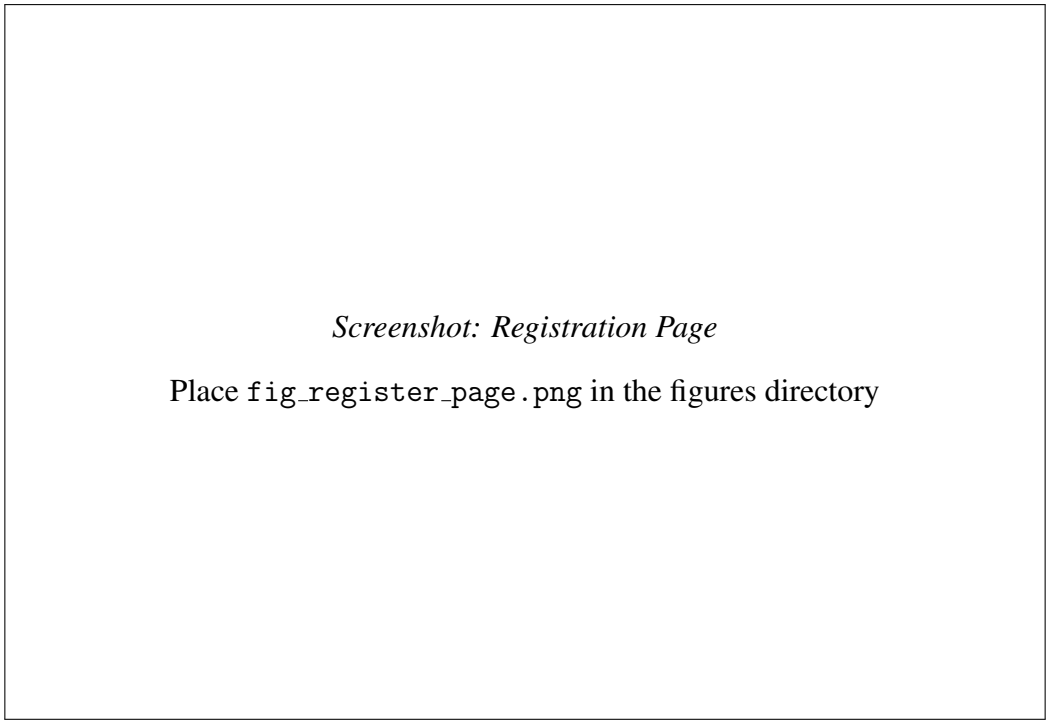

Figure 4.2: User Registration Page

4.3.3 Login Page

The login page supports two authentication methods: email and password credentials, and Sui wallet-based signature authentication. The wallet login option is presented alongside the traditional form-based login.

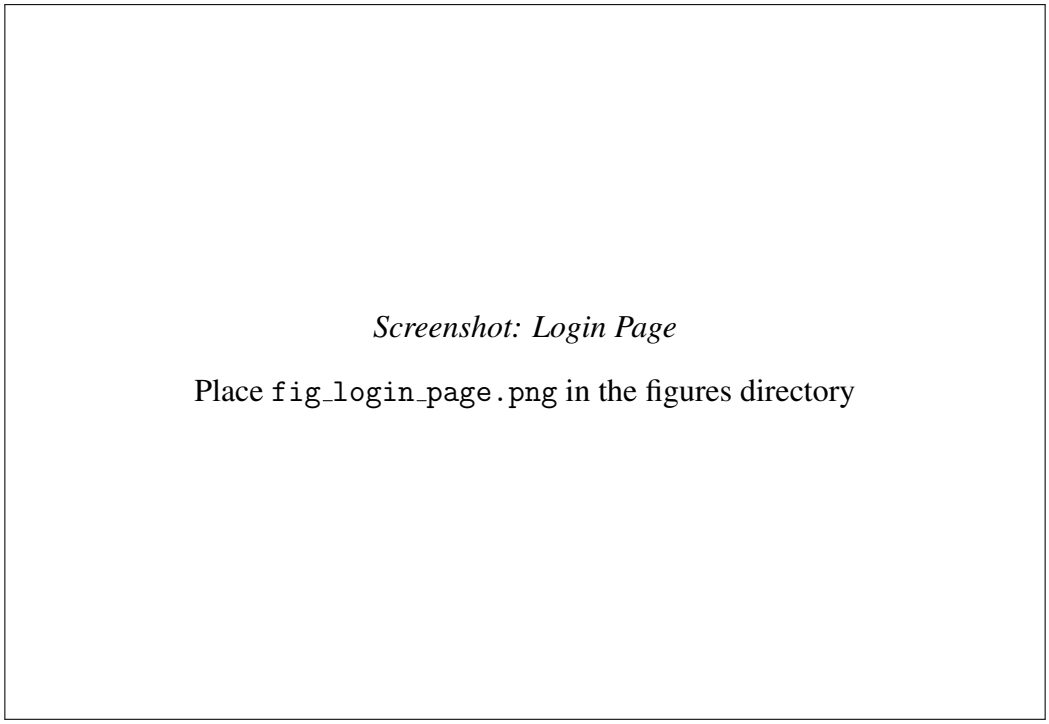

Figure 4.3: Login Page with Dual Authentication

4.3.4 Farmer Dashboard

The farmer dashboard provides an overview of the farmer's current status, including recent telemetry readings, active produce batches, yield prediction summaries, and recent activity notifications.

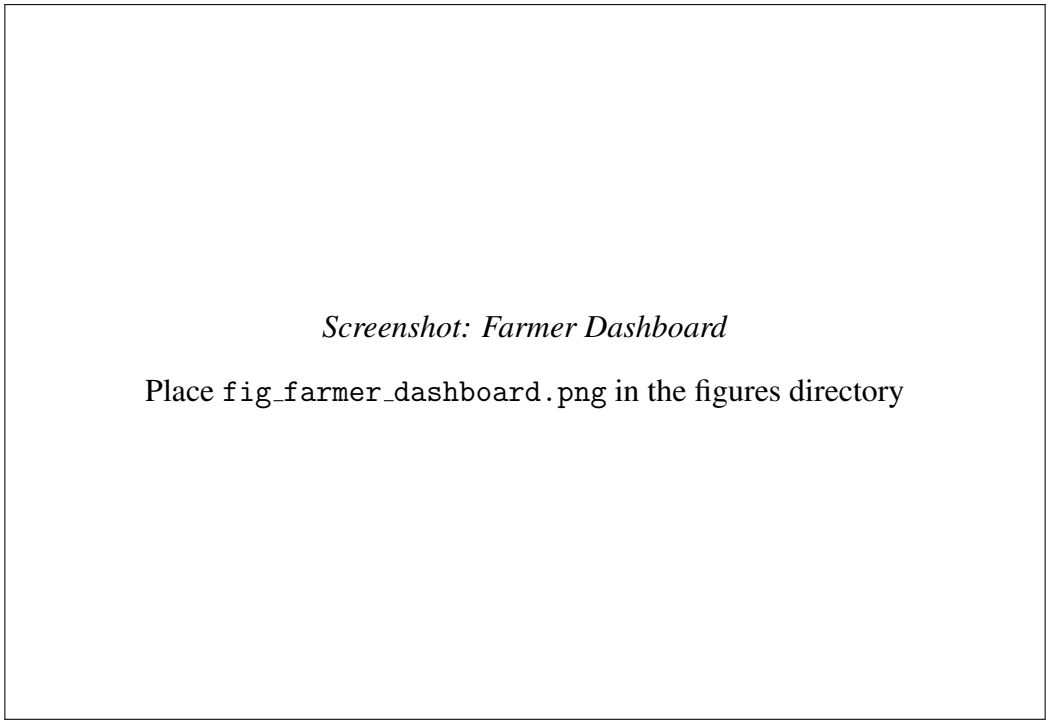

Figure 4.4: Farmer Dashboard

4.3.5 Telemetry Submission Page

This page allows farmers to enter environmental readings. The form fields accept temperature, soil moisture percentage, and soil pH values. Upon submission, the system computes the SHA-256 hash and displays the confirmation along with the generated hash value.

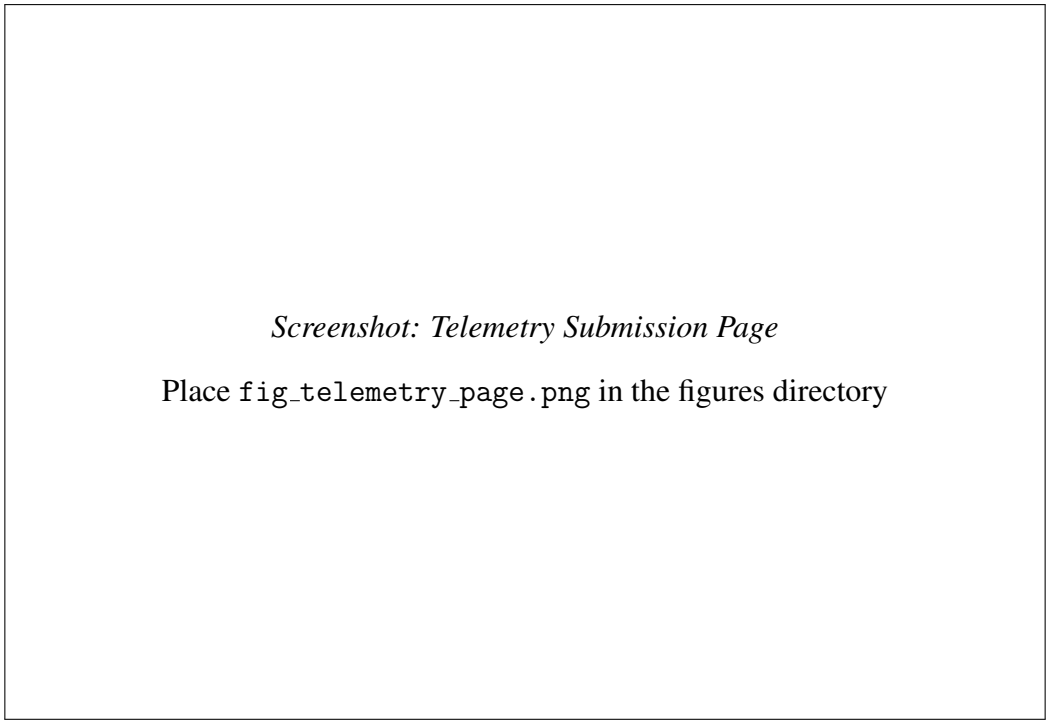

Figure 4.5: Telemetry Submission Page

4.3.6 Batch Minting Interface

The mint batch page guides the farmer through the two-step batch creation process. The farmer selects a telemetry record, enters the crop type and weight, and initiates the backend preparation. The interface then connects to the Sui wallet for on-chain minting and confirms the transaction upon completion.

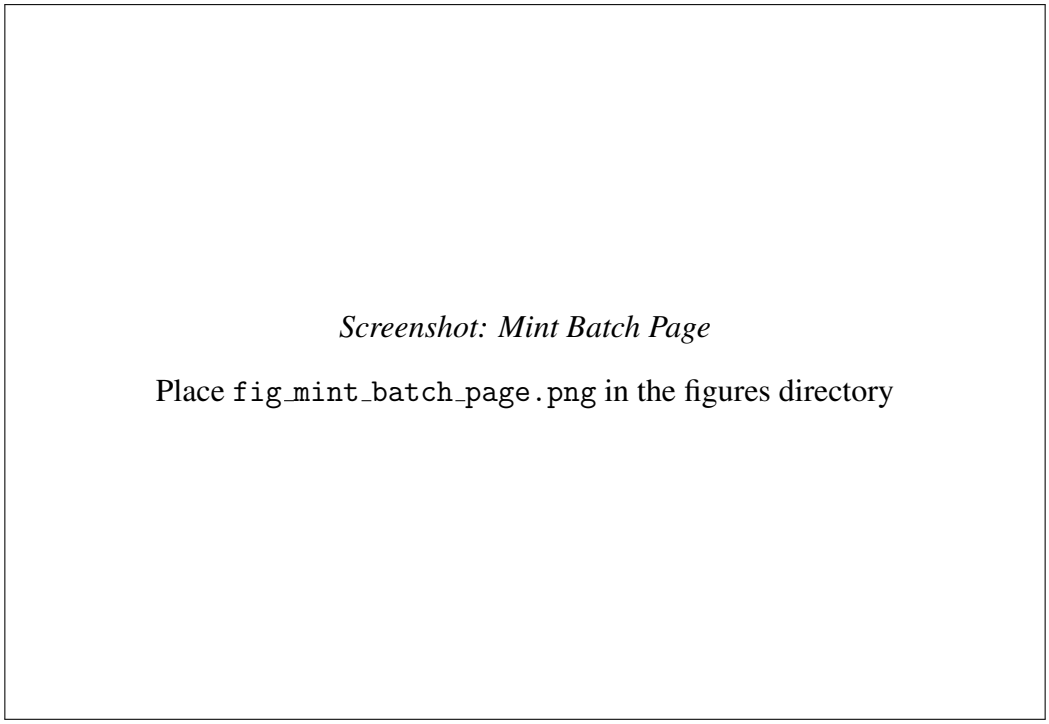

Figure 4.6: Produce Batch Minting Interface

4.3.7 Yield Prediction View

The yield prediction page displays the computed forecast including the predicted yield in metric tons, the confidence score, a variance index, the number of data points analysed, and a contextual recommendation for the farmer.

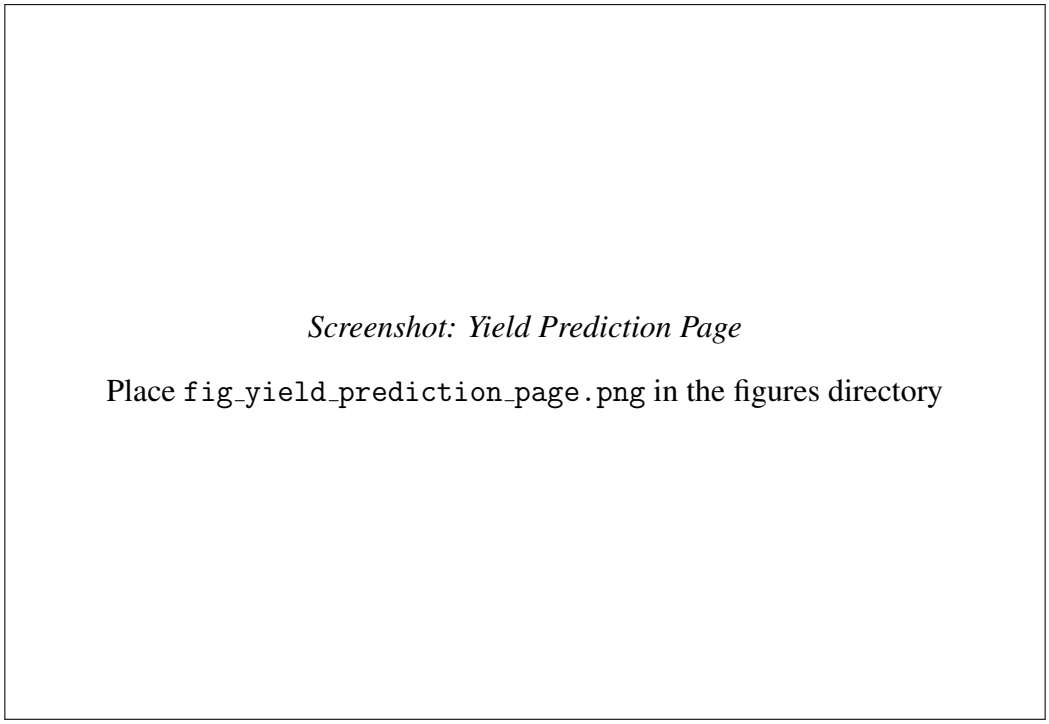

Figure 4.7: Yield Prediction Results Page

4.3.8 Logistics Dashboard

The logistics handler dashboard shows a summary of batches under the handler's custody, pending transfers, shipment statuses, and tracking information.

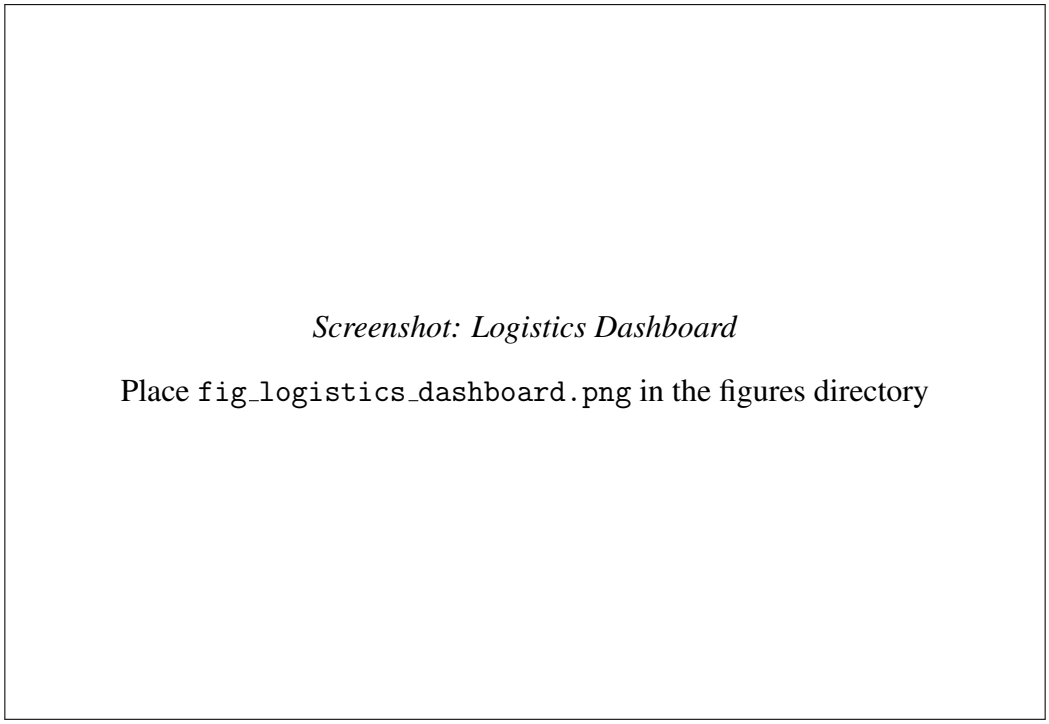

Figure 4.8: Logistics Handler Dashboard

4.3.9 Custody Transfer Page

The transfer page enables logistics handlers to initiate and receive custody transfers of produce batches. The interface shows the batch details, the current custodian, and the target recipient, with the transfer being recorded both in the backend database and on the Sui blockchain.

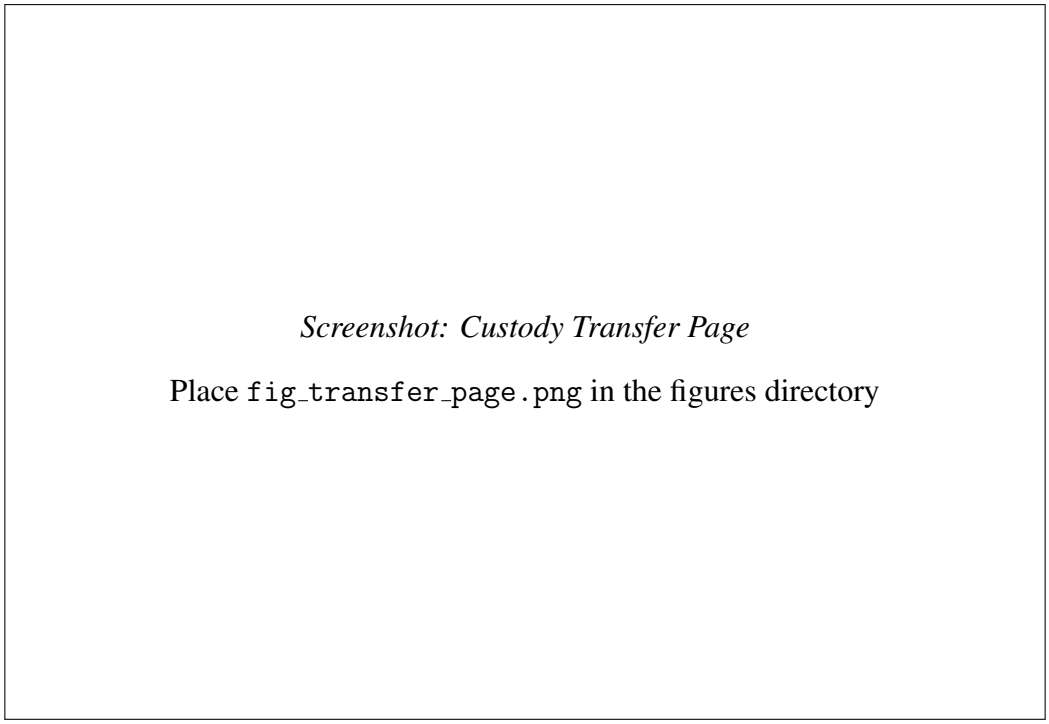

Figure 4.9: Custody Transfer Page

4.3.10 Administrator Dashboard

The administrator dashboard provides a system-wide view including total user counts by role, batch statistics by status, and system health indicators.

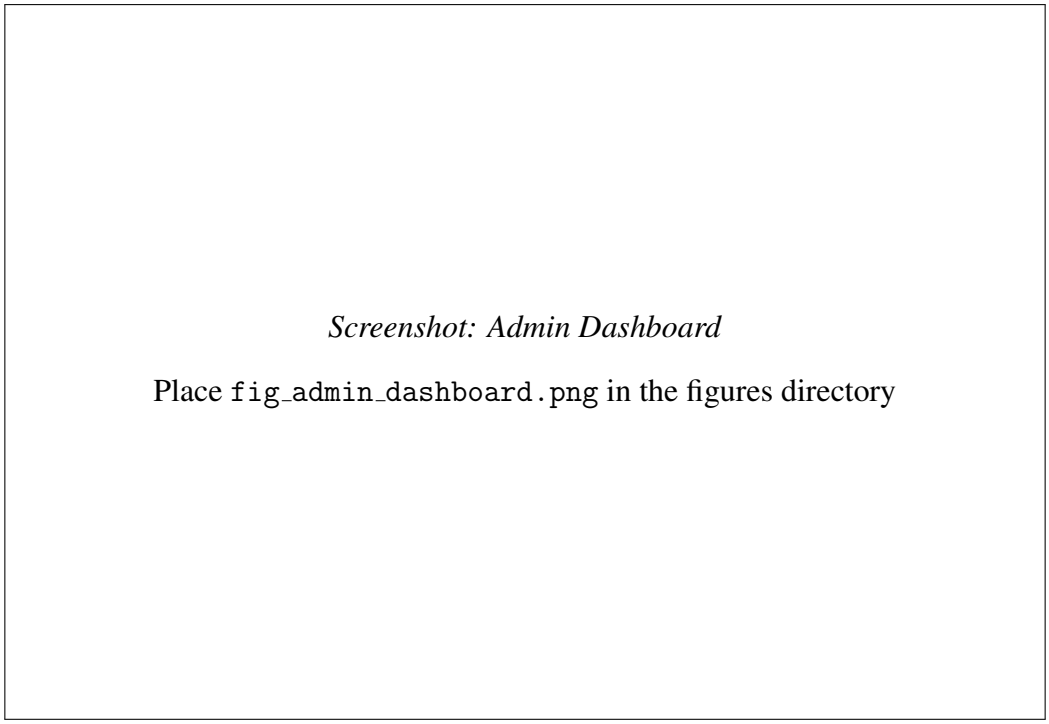

Figure 4.10: Administrator Dashboard

4.3.11 Audit Logs Page

The audit logs page displays a chronological record of system activities, including user actions, resource modifications, and timestamps. This feature supports accountability and post-incident analysis.

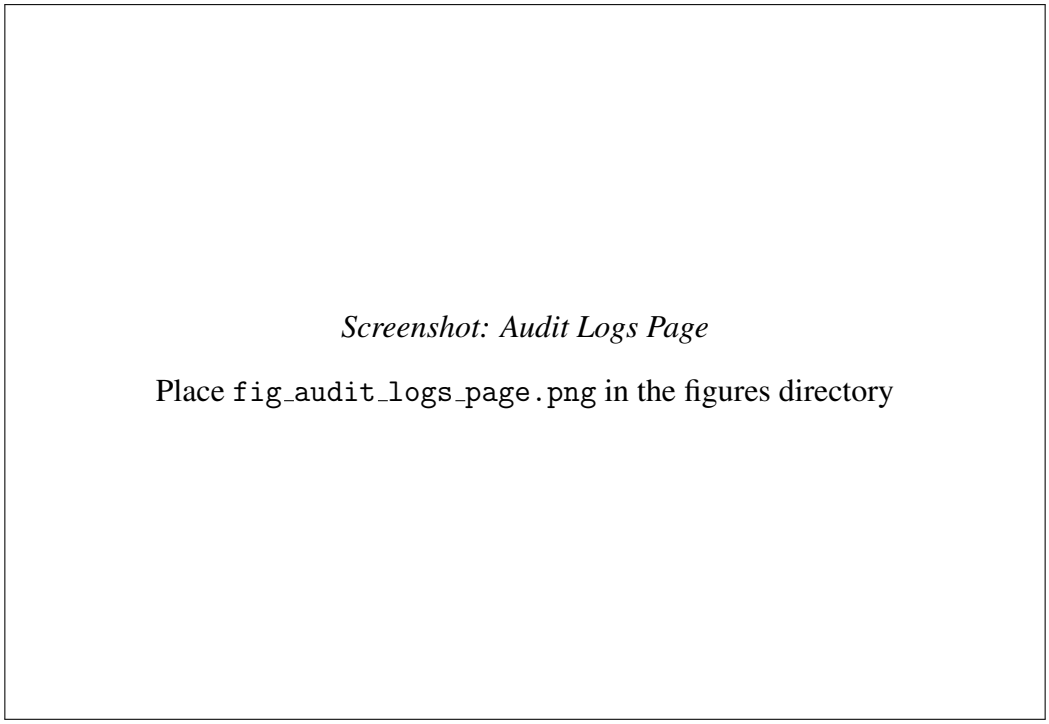

Figure 4.11: Audit Logs Page

4.4 Key Algorithm Implementations

This section presents the core algorithms implemented in TerraNode, with code excerpts and explanations.

4.4.1 SHA-256 Data Integrity Hashing

The integrity hashing function generates a deterministic SHA-256 digest from the core fields of a telemetry record. The use of canonical JSON serialization with sorted keys, no whitespace, and fixed-precision formatting ensures that the same input data always produces the same hash, regardless of the order in which the fields are processed.

```
1 import hashlib
2 import json
3
4 def generate_telemetry_hash(farmer_id, recorded_at,
5                              temperature, soil_moisture,
6                              soil_ph):
```

```
    canonical_data = json.dumps({
        "farmer_id": str(farmer_id),
        "recorded_at": recorded_at.isoformat(),
        "temperature_celsius": f"{temperature:.2f}",
        "soil_moisture_percentage": f"{soil_moisture:.2f}",
        "soil_ph": f"{soil_ph:.2f}",
    }, sort_keys=True, separators=(",", ":"))

    return hashlib.sha256(
        canonical_data.encode("utf-8")
    ).hexdigest()
```

Listing 4.1: Telemetry Data Integrity Hash Generation

The function in Listing 4.1 accepts the farmer identifier, recording timestamp, and three environmental measurements. It constructs a JSON string with sorted keys and minimal separators, then applies the SHA-256 algorithm to produce a 64-character hexadecimal digest. This hash serves as a fingerprint for the telemetry record: any modification to any field, no matter how small, would produce a completely different hash value.

4.4.2 Sui Ed25519 Signature Verification

The wallet-based login mechanism verifies that the user possesses the private key corresponding to their registered Sui public key. The verification algorithm reconstructs the signed payload using the Sui personal message signing protocol and checks the Ed25519 signature.

```
import base64
import hashlib
from nacl.signing import VerifyKey

def verify_sui_personal_message(message_str,
                                signature_b64):
    sig_bytes = base64.b64decode(signature_b64)
```

```
    flag = sig_bytes[0]  # 0 = Ed25519
    signature = sig_bytes[1:65]
    pubkey = sig_bytes[65:]

    # Sui personal message intent prefix
    intent_prefix = bytes([3, 0, 0])
    msg_bytes = message_str.encode('utf-8')

    # ULEB128 length encoding
    def to_uleb128(n):
        result = bytearray()
        while True:
            byte = n & 0x7f
            n >>= 7
            if n != 0:
                byte |= 0x80
            result.append(byte)
            if n == 0:
                break
        return bytes(result)

    bcs_msg = to_uleb128(len(msg_bytes)) + msg_bytes
    intent_message = intent_prefix + bcs_msg

    # BLAKE2b hash with 32-byte digest
    hashed_msg = hashlib.blake2b(
        intent_message, digest_size=32
    ).digest()

    # Ed25519 verification
    verify_key = VerifyKey(pubkey)
```

```
    verify_key.verify(hashed_msg, signature)
    return True
```

Listing 4.2: Sui Personal Message Signature Verification

The function in Listing 4.2 implements the full Sui personal message verification protocol. The signature bytes are decoded from base64 and split into the scheme flag, the 64-byte Ed25519 signature, and the public key. The message is prepended with the Sui intent prefix and BCS-encoded before being hashed with BLAKE2b. The final verification is performed by the PyNaCl library's `VerifyKey` class.

4.4.3 Yield Prediction Algorithm

The prediction algorithm implements the weighted moving average model described in Chapter Three. The following listing shows the core computation logic.

```
def predict_yield(farmer_id):
    records = EnvironmentalTelemetry.objects.filter(
        farmer_id=farmer_id
    ).order_by("-recorded_at")[:90]

    if len(records) < 5:
        return {"error": "Insufficient data points"}

    avg_temp = sum(
        r.temperature_celsius for r in records
    ) / len(records)
    avg_moisture = sum(
        r.soil_moisture_percentage for r in records
    ) / len(records)
    avg_ph = sum(
        r.soil_ph for r in records
    ) / len(records)
```

```
18
19      predicted_yield = (50.0
20          + (avg_moisture * 0.5)
21          - abs(22.0 - avg_temp) * 2.0)
22      predicted_yield = max(0.0, predicted_yield)
23
24      confidence = min(0.95, len(records) / 90.0)
25
26      recommendation = "Maintain current conditions."
27      if avg_moisture < 40:
28          recommendation = ("Increase irrigation to "
29                            "prevent crop stress.")
30      elif avg_temp > 35:
31          recommendation = ("Temperature is too high, "
32                            "consider shading.")
33
34      return {
35          "confidence_score": confidence,
36          "predicted_yield_metric_tons": predicted_yield,
37          "historical_variance_index": 0.15,
38          "data_points_analyzed": len(records),
39          "recommendation": recommendation,
40          "averages": {
41              "temp": avg_temp,
42              "moisture": avg_moisture,
43              "ph": avg_ph
44          }
45      }
```

Listing 4.3: Crop Yield Prediction Service

The function in Listing 4.3 retrieves up to 90 recent telemetry records, calculates the mean

values of temperature, soil moisture, and pH, applies the yield formula from Equation 3.2, computes the confidence score from Equation 3.3, and generates a contextual recommendation based on the computed averages.

4.5 System Evaluation

4.5.1 Functional Testing Results

The system was tested against the functional requirements defined in Chapter Three. Table 4.2 summarizes the test cases and their outcomes.

Table 4.2: Functional Test Results

Req.	**Test Case**	**Expected**	**Result**
FR-01	Register with valid email, password, name, and role	Account created	Pass
FR-02	Login with valid credentials	JWT tokens returned	Pass
FR-02	Access farmer endpoint as logistics user	403 Forbidden	Pass
FR-03	Wallet login with valid Ed25519 signature	JWT tokens returned	Pass
FR-03	Wallet login with invalid signature	401 Unauthorized	Pass
FR-04	Submit telemetry with valid ranges	Record created with hash	Pass
FR-04	Submit telemetry with out-of-range pH (15.0)	400 Bad Request	Pass
FR-05	Verify SHA-256 hash matches recomputed value	Hashes match	Pass
FR-06	Prepare batch linked to telemetry record	Batch in PENDING status	Pass
FR-07	Confirm batch with Sui object ID	Status changes to MINTED	Pass
FR-08	Transfer custody to logistics handler	Custodian updated	Pass
FR-09	Request prediction with 10 data points	Yield and confidence returned	Pass
FR-09	Request prediction with 3 data points	Error returned	Pass
FR-10	Farmer sees farmer-specific dashboard	Correct dashboard rendered	Pass
FR-10	Admin sees all system data	Full data access confirmed	Pass

All functional requirements were verified through a combination of automated unit tests and manual integration testing. The backend test suite includes 143 automated test cases covering

the users, telemetry, ledger, and analytics modules.

4.5.2 Security Evaluation

The security properties of TerraNode were evaluated across four dimensions:

Password Security: User passwords are hashed using the Argon2id algorithm, which is resistant to both side-channel and brute-force attacks. Argon2id was selected as the primary hasher in the Django settings, with PBKDF2 configured as a fallback for compatibility. The stored hash values were verified to be irreversible through manual inspection.

JWT Token Lifecycle: Access tokens expire after 15 minutes by default, limiting the window of exposure if a token is compromised. Refresh tokens are rotated on each use, and the old tokens are blacklisted in the database. Expired and blacklisted tokens were confirmed to be rejected by the backend.

Rate Limiting: The login endpoint is protected by a custom throttle class that limits authentication attempts to five per minute per client. Exceeding this limit was confirmed to return a 429 Too Many Requests response.

Data Integrity: The SHA-256 hash verification service was tested by computing the hash of a telemetry record, modifying a single field by 0.01, and confirming that the recomputed hash differed from the original. This confirms that even minor data modifications are detectable.

4.5.3 Blockchain Performance Metrics

Table 4.3 presents the blockchain performance metrics observed during testing on the Sui Testnet. These measurements were taken over a series of 20 transactions of each type.

Table 4.3: Sui Testnet Performance Metrics

Operation	**Avg. Latency**	**Avg. Gas Cost**	**Success Rate**
Batch Minting (mint_batch)	$\sim$1.2s	$\sim$0.003 SUI	100%
Custody Transfer (transfer_custody)	$\sim$0.9s	$\sim$0.002 SUI	100%
Transaction Finality	$<$3s	—	—

The observed latencies are well within acceptable bounds for agricultural supply chain operations, where batch creation and transfer events typically occur on an hourly or daily basis rather than in real-time. The low gas costs demonstrate the feasibility of using the Sui blockchain for routine agricultural data recording without incurring prohibitive transaction fees.

4.5.4 Prediction Accuracy Assessment

Since the system uses its own generated telemetry data rather than an external benchmark dataset, the prediction accuracy was assessed through consistency and sensitivity analysis rather than against ground-truth yield values.

Consistency Test: When the same set of telemetry records was used as input, the prediction engine consistently produced identical output values. This confirms the deterministic nature of the algorithm and the correctness of the caching mechanism.

Sensitivity Analysis: The predicted yield was computed for a range of average temperature and moisture values to verify that the model responds correctly to changes in input conditions. Table 4.4 shows representative results.

Table 4.4: Yield Prediction Sensitivity Analysis

Avg. Temp (°C)	**Avg. Moisture (%)**	**Predicted Yield (MT)**	**Recommendation**
22.0	60.0	80.0	Maintain current conditions
22.0	30.0	65.0	Increase irrigation
35.0	60.0	54.0	Temperature too high, consider shading
10.0	60.0	56.0	Maintain current conditions
40.0	20.0	24.0	Increase irrigation

The results confirm that the model correctly increases yield predictions with higher moisture levels, penalizes temperature deviations from the optimal 22 degrees, and generates appropriate recommendations based on the computed averages.

4.5.5 API Response Time Benchmarks

Table 4.5 presents the average API response times observed during local testing. These measurements reflect the performance of the Django backend running in development mode on a standard workstation.

Table 4.5: API Response Time Benchmarks

Endpoint	**Method**	**Avg. Response Time**
/api/v1/auth/login/	POST	∼120ms
/api/v1/telemetry/submit/	POST	∼85ms
/api/v1/telemetry/history/	GET	∼45ms
/api/v1/ledger/prepare/	POST	∼95ms
/api/v1/ledger/list/	GET	∼40ms
/api/v1/analytics/predict/ (cached)	GET	∼15ms
/api/v1/analytics/predict/ (uncached)	GET	∼110ms

All API endpoints respond well within the 500-millisecond threshold defined in the non-functional requirements. The significant difference between cached and uncached prediction responses (15ms versus 110ms) demonstrates the effectiveness of the Redis caching strategy.

4.6 Chapter Summary

This chapter documented the implementation of the TerraNode system across its three main components: the Django REST backend, the React TypeScript frontend, and the Sui Move smart contract. The user interface was presented through eleven screen descriptions, each corresponding to a key functional area of the platform. Three core algorithms were documented with code listings: the SHA-256 data integrity hash generation, the Sui Ed25519 signature verification protocol, and the weighted moving average yield prediction service. The system evaluation confirmed that all ten functional requirements are satisfied, that the security mechanisms operate as designed, that blockchain transactions complete within acceptable latency

thresholds on the Sui Testnet, and that the prediction engine responds correctly to variations in input conditions. These findings are synthesized in the following chapter.

CHAPTER FIVE

SUMMARY, CONCLUSION, AND RECOMMENDATIONS

5.1 Summary of Findings

This project set out to design and implement a secure, analytics-driven agricultural data management system using blockchain technology. The work was guided by four specific objectives, each of which has been addressed through the design, implementation, and evaluation presented in the preceding chapters.

Objective (i) — Web-Based Agricultural Data Management Platform: A modular web interface was designed and implemented using React and TypeScript for the frontend and Django REST Framework for the backend. The platform captures environmental telemetry data, including temperature in degrees Celsius, soil moisture percentage, and soil pH, through validated input forms. The data is stored in a PostgreSQL database and visualized through role-specific dashboards tailored to the needs of farmers, logistics handlers, and system administrators. The interface supports both credential-based and Sui wallet-based authentication, accommodating users with varying levels of blockchain familiarity.

Objective (ii) — Analytics Module for Agricultural Forecasting: A predictive analytics engine was developed and integrated into the backend. The engine applies a weighted moving average model to historical telemetry data, computing yield forecasts based on the mean values of temperature, soil moisture, and soil pH. The prediction formula accounts for moisture availability as a positive factor and penalizes deviations from an optimal growing temperature of 22 degrees Celsius. A confidence score is computed based on the number of available data points, and contextual recommendations are generated to guide the farmer's decision-making. Results

are cached in Redis to ensure responsive performance.

Objective (iii) — Blockchain-Based Security Mechanisms: Data integrity is ensured through a multi-layered approach. Every telemetry record is protected by a SHA-256 hash computed from a canonical JSON representation of its core fields. Produce batch records are minted as programmable objects on the Sui blockchain using a Move smart contract, which records the crop type, weight, originating farmer, current custodian, and the telemetry integrity hash on-chain. Custody transfers between stakeholders are enforced by the smart contract, which verifies that only the current custodian can initiate a transfer. The blockchain anchoring creates a tamper-resistant audit trail that links each on-chain asset to its off-chain data source.

Objective (iv) — Performance Evaluation: The system was evaluated across multiple dimensions. Functional testing confirmed that all ten functional requirements are satisfied. Security evaluation verified the effectiveness of Argon2id password hashing, JWT token lifecycle management, rate limiting, and data integrity hash verification. Blockchain performance testing on the Sui Testnet showed average minting latency of approximately 1.2 seconds and transfer latency of approximately 0.9 seconds, with 100 percent success rates and minimal gas costs. Prediction accuracy was assessed through consistency and sensitivity analysis, confirming that the model responds correctly to variations in input conditions and produces deterministic results.

5.2 Conclusion

The TerraNode project demonstrates that it is technically feasible to build an integrated agricultural data management platform that unifies secure data storage, blockchain-based immutability, supply chain traceability, and predictive analytics within a single, cohesive architecture. The literature review in Chapter Two revealed that existing systems typically address only one or two of these capabilities in isolation. TerraNode bridges this gap by making blockchain verification a prerequisite for analytical processing, ensuring that yield predictions are derived from data whose integrity can be mathematically verified.

The choice of the Sui blockchain proved to be well-suited for this application. Unlike

traditional public blockchains that impose high transaction fees and long confirmation times, Sui's delegated proof-of-stake consensus and object-centric data model enabled low-cost, sub-second transaction finality for both batch minting and custody transfer operations. The Move programming language's linear type system provided additional safety guarantees at the smart contract level, preventing common vulnerabilities such as double-spending of batch objects.

The three-tier architecture, separating the React frontend, Django backend, and Sui blockchain into distinct layers, facilitated a clean separation of concerns and allowed each component to be developed and tested independently. The role-based access control system ensures that each stakeholder type interacts only with the functionality relevant to their role, reducing complexity and potential security risks.

While the current analytics engine uses a simplified weighted moving average model, the architecture is designed to accommodate more sophisticated prediction algorithms in the future. The modular service layer can be extended to incorporate machine learning models trained on larger datasets without requiring changes to the API interface or the frontend.

In summary, TerraNode contributes a working prototype that addresses a genuine gap in the agricultural technology landscape. It shows that the combination of modern web technologies, a next-generation blockchain platform, and data-driven analytics can produce a system that is both technically robust and practically accessible to the smallholder farming communities it is intended to serve.

5.3 Recommendations for Future Work

Based on the experience gained during the development and evaluation of TerraNode, the following recommendations are made for future enhancements:

1. **IoT Sensor Integration:** The current system relies on manually entered telemetry data. Future iterations should integrate directly with physical IoT sensors deployed in agricultural fields, enabling automated and continuous data collection. This would eliminate the potential for human error in data entry and increase the volume and frequency of telemetry records available for analytics.

2. **Advanced Machine Learning Models:** The weighted moving average model provides a functional starting point, but more sophisticated approaches such as Random Forests, Long Short-Term Memory (LSTM) networks, or the CNN-RNN framework proposed by Khaki et al. (2019) could significantly improve prediction accuracy. Training these models on larger, geographically diverse datasets would also enhance their generalizability.

3. **Sui Mainnet Deployment:** The current implementation operates on the Sui Testnet, which does not carry real economic value. Deploying the smart contract on the Sui Mainnet would enable genuine asset tracking and could facilitate integration with decentralized finance (DeFi) protocols for agricultural lending and insurance.

4. **Mobile Application:** Developing a mobile companion application for Android and iOS would improve accessibility for farmers who primarily access the internet through smartphones. The mobile application could also leverage device sensors (GPS, camera) to augment telemetry data with location and visual crop condition information.

5. **Multi-Language Support:** The current interface is available only in English, which limits its accessibility in regions where English is not the primary language. Adding support for local languages would significantly broaden the potential user base.

6. **Storage Optimization:** As the volume of on-chain data grows, storage optimization techniques such as the Storage Light Node model proposed by Sun et al. (2025) could be adopted to reduce the resource requirements for nodes participating in the TerraNode network.

7. **Interoperability Standards:** Future work should explore the adoption of standardized agricultural data formats and APIs to enable TerraNode to interoperate with other agricultural information systems, weather services, and market platforms.

5.4 Chapter Summary

This chapter summarized the findings of the TerraNode project against its four specific objectives, drew conclusions about the technical contributions and practical implications of the

work, and outlined seven concrete recommendations for future development. The project has successfully demonstrated that blockchain-secured agricultural data management, combined with predictive analytics and role-based web interfaces, can be realized within a unified platform built on modern web technologies and the Sui blockchain.

REFERENCES

AgTrust Consortium. (2026). Agtrust: Framework for securing critical agriculture technology and emerging solutions. *IEEE Access*, *14*, 1–16.

Al Layek, M. A., Kamal, M. S., & Hasan, M. F. (2025). A secured triad of iot, machine learning, and blockchain for crop forecasting in agriculture. *arXiv preprint arXiv:2505.01196*.

Al-Absi, A. A., Al-Absi, M. A., & Lee, J. (2023). Blockchain-based smart farm security framework for the internet of things. *Sensors*, *23*(18), 7992.

Caro, M. P., Ali, M. S., Vecchio, M., & Giaffreda, R. (2018). Blockchain-based traceability in agri-food supply chain management: A practical implementation. *2018 IoT Vertical and Topical Summit on Agriculture – Tuscany (IOT Tuscany)*, 1–4. https://doi.org/10.1109/IOT-TUSCANY.2018.8373021

Chauhan, V., Jain, N., Waghmare, M., Parmar, A., Nandeha, N., Trivedi, A., Sharma, N., & Rahman, S. W. (2025). Blockchain and big data analytics in agriculture: A review of digital innovations. *Journal of Experimental Agriculture International*, *47*(11), 58–68. https://doi.org/10.9734/jeai/2025/v47i113849

Cocco, L., Tonelli, R., & Marchesi, M. (2021). Blockchain and self sovereign identity to support quality in the food supply chain. *Future Internet*, *13*(12), 301.

Dzreke, S. S. (2025). Securing the connected farm: Cyber threats and resilient architectures for the internet of agrithings. *Engineering Science & Technology Journal*, *6*(11), 642–659.

Garg, B., Aggarwal, S., & Sokhal, J. (2018). Crop yield forecasting using fuzzy logic and regression model. *Computers and Electrical Engineering*, *67*, 383–403. https://doi.org/10.1016/j.compeleceng.2017.11.015

Khaki, S., Wang, L., & Archontoulis, S. V. (2019). A cnn-rnn framework for crop yield prediction. *Frontiers in Plant Science*, *10*, 1750. https://doi.org/10.3389/fpls.2019.01750

Ktari, J., Frikha, T., Chaabane, F., Hamdi, M., & Hamam, H. (2022). Agricultural lightweight embedded blockchain system: A case study in olive oil. *Electronics*, *11*(20), 3394. https://doi.org/10.3390/electronics11203394

Lin, Y.-P., Petway, J. R., Anthony, J., Mukhtar, H., Liao, S.-W., Chou, C.-F., & Ho, Y.-F. (2017). Blockchain: The evolutionary next step for ict e-agriculture. *Environments*, *4*(3), 50. https://doi.org/10.3390/environments4030050

Liu, J., Zhang, H., & Zhen, L. (2021). Blockchain technology in maritime supply chains: Applications, architecture and challenges. *International Journal of Production Research*, *61*(11), 3547–3563. https://doi.org/10.1080/00207543.2021.1930239

Mirabelli, G., & Ferraro, V. (2020). Blockchain and agricultural supply chains traceability: Research trends and future challenges. *Procedia Manufacturing*, *42*, 414–421. https://doi.org/10.1016/j.promfg.2020.02.054

Mysten Labs. (2024). Sui blockchain documentation: Architecture and move programming [Accessed: 2026-07-15].

Padhiary, M. (2025). Emerging technologies for smart and sustainable precision agriculture. *Discover Robotics*, *1*(6). https://doi.org/10.1007/s44430-025-00006-0

Rahman, M. S., Hossain, M. S., Rahman, M. K., Islam, M. R., Sumon, M. F. I., Siam, M. A., & Debnath, P. (2025). Enhancing supply chain transparency with blockchain: A data-driven analysis of distributed ledger applications. *Journal of Business and Management Studies*, *7*(4), 59–77. https://doi.org/10.32996/jbms

Raj, M., Gupta, S., Chamola, V., Elhence, A., Garg, T., Atiquzzaman, M., & Niyato, D. (2022). A survey on smart agriculture: Development modes, technologies, and security and privacy challenges. *IEEE Access*, *10*, 4719–4732. https://doi.org/10.1109/ACCESS.2021.3137083

Sharma, A., Sharma, A., Bhatia, T., & Singh, R. K. (2023). Blockchain enabled food supply chain management: A systematic literature review and bibliometric analysis. *Operations Management Research*, *16*(3), 1594–1618.

Sun, M., Luo, N., Bin, X., Chen, F., Liu, Q., Liu, X., & Sun, C. (2025). Data storage and query optimization for blockchain-based agricultural supply chains using storage light nodes. *Scientific Reports*, *15*, 17258. https://doi.org/10.1038/s41598-025-17258-w